

- peer-to-peer computing, 43-44
- real-time embedded systems, 45-46
- traditional, 40-41
- virtualization, 34-35
- concurrency, 163**
- Concurrency Runtime (ConcRT), 241-242, 890**
- concurrent dispatch queue, 185**
- conditional-wait construct, 281**
- condition variables, 278, 279, 302-303, 309-311, 889**
- confidentialit , breach of, 622**
- confinemen problem, 678**
- conflic phase (of dispatch latency), 229**
- conflict-resolutio mechanism (Linux), 784, 785**
- congestion control, 749**
- Connected Standby, 837**
- connectionless protocols, 747**
- connectionless (UDP) sockets, 147**
- connection-oriented protocols, 748**
- connection-oriented (TCP) sockets, 147**
- connection ports, 138**
- consistency, of distributed fil systems, 767**
- consistency checker, 586-587**
- consistency checking, 586-587**
- consistency semantics, 608-609**
- consolidation, 706**
- constant angular velocity (CAV), 457**
- constant linear velocity (CLV), 456-457**
- consumer process, 126-127, 290, 291, 559-560**
- containers, 592, 718, 719**
- container objects (Windows 10), 664**
- containment, application, 703, 718-719**
- contaminants, 344**
- contended locks, 271**
- content-addressable storage, 484**
- contention scope, 217-218**
- context (of process), 114**
- context (of thread), 194**
- context switches, 114-115, 204**
- contiguous allocation, 356-360, 570-573**
- contiguous bit, 432-433**
- Control-flo Enforcement Technology (CET), 828**
- Control-Flow Guard (CFG), 827**
- controlled access, 552-554**
- controller(s), 456**
 - defined, 491
 - device, 456
 - direct-memory-access, 498
 - fibre channel bus, 491
 - host, 456
- control partitions, 714**
- control programs, 5**
- control register, 492**
- convenience, 1**
- convoy effect, 207**
- cooperating processes, 123, 257**
- cooperative scheduling, 202**
- coordination, among processes, 260**
- copy-on-write technique, 399-401, 853**
- copy rights, access matrix with, 677**
- copy semantics, 510**
- core(s), 15-16, 18**
 - big and little, 226-227
 - dual-core design, 17, 18
 - multicore processors, 221-224
 - multicore programming, 162-166
 - multicore systems, 16-18
 - scheduling processes to run on, 199
- core dump, 95-96**
- core frameworks (macOS and iOS), 87**
- CoreUI, 825**
- counts, 533, 534**
- counters, 96-97**
 - LRU page replacement with, 408
 - program, 27, 106, 109
 - timestamp, 845
- counting, 580**
- counting-based page replacement algorithm, 411-412**
- counting semaphore, 273**
- C program, memory layout in, 108**
- CPUs (central processing units), 18, 318**
- CPU-bound processes, 112**
- CPU burst, 201**
- CPU-I/O burst cycle, 201**
- CPU registers, 110**
- CPU scheduler, 113-114, 201**
- CPU scheduling, 24, 199-251**
 - about, 201
 - algorithms for, 205-217
 - evaluation of, 244-249
 - first-come, first-served scheduling of, 206-207
 - multilevel feedback-queue scheduling of, 216-217
 - multilevel queue scheduling of, 214-216
 - priority scheduling of, 211-214
 - round-robin scheduling of, 209-211
 - shortest-job-first scheduling of, 207-209
 - criteria, 204-205
 - dispatcher, role of, 203-204

- and I/O-CPU burst cycle, 201
 - multi-processor scheduling, 220-227
 - approaches to, 220-221
 - heterogeneous multiprocessing, 226-227
 - and load balancing, 224-225
 - and multicore processors, 221-224
 - and processor affinity, 225-226
 - operating-system examples, 234-244
 - Linux scheduling, 234-239
 - Solaris scheduling, 242-244
 - Windows scheduling, 239-242
 - preemptive vs. nonpreemptive scheduling, 202-203
 - real-time, 227-234
 - earliest-deadline-first scheduling, 232-233
 - and minimizing latency, 227-229
 - POSIX real-time scheduling, 233-234
 - priority-based scheduling, 229-230
 - proportional share scheduling, 233
 - rate-monotonic scheduling, 230-232
 - thread scheduling, 217-219
 - virtual machines, 720
 - CPU-scheduling information (PCBs), 110**
 - CPU utilization, 204**
 - crashes, 96**
 - crash dumps, 96**
 - CRCs (cyclic redundancy checks), 462**
 - creation:**
 - of files, 532, 542
 - of processes, 116-121
 - credentials, 787**
 - critical sections, 260**
 - critical-section object, 297, 888**
 - critical-section problem, 260-270**
 - Peterson's solution to, 262-265
 - and semaphores, 272-276
 - and synchronization hardware, 265-270
 - cryptography, 637-648**
 - defined, 638
 - and encryption, 638-645
 - asymmetric encryption, 641
 - authentication, 641-644
 - key distribution, 644-645
 - symmetric encryption, 639-640
 - implementation of, 645-646
 - TLS example of, 646-648
 - CSC (client-side caching), 883**
 - C-SCAN scheduling, 460**
 - C-SCAN scheduling algorithm, 460**
 - C shell, 58**
 - ctfs (fil system), 598**
 - cumulative ACK, 748**
 - current directory, 546**
 - current-file-position pointer, 532**
 - cycle stealing, 499**
 - cyclic redundancy checks (CRCs), 462**
 - cylinder (hard disk drive), 450**
 - cylinder groups, 806**
- ## D
- d (page offset), 360**
 - DAC (discretionary access control), 684**
 - daemons, 22, 781**
 - daemon processes, 690**
 - daisy chain, 490**
 - DAM (Desktop Activity Moderator), 837**
 - dark web, 634**
 - Darwin operating system, 85, 88, 687-688**
 - data attributes, 875**
 - databases, 341, 842, 856**
 - data dependency, 164**
 - data-encryption standard (DES), 639**
 - Data Execution Prevention (DEP), 827**
 - datagrams, 743**
 - data-in register, 492**
 - data-link layer, 742**
 - data-link layer protocol, 645**
 - data loss, mean time of, 474**
 - data migration, 751-752**
 - data-out register, 492**
 - data parallelism, 165, 166**
 - data passing, between processes, 813**
 - data section (of process), 106**
 - data splitting, 164**
 - data striping, 475**
 - data view attribute, 862**
 - DCOM, 882**
 - DDoS attacks, 636**
 - deadline scheduler, 460, 461**
 - deadlock(s), 283-284, 317-343**
 - avoidance of, 326, 330-337
 - with banker's algorithm, 333-337
 - with resource-allocation-graph algorithm, 333
 - with safe-state algorithm, 331-333
 - characterization, 321-326
 - defined, 317
 - detection of, 337-341
 - methods for handling, 326-327
 - in multithreaded applications, 319-321
 - necessary conditions for, 321-323
 - prevention of, 326-330
 - recovery from, 341-343
 - system model for, 318-319

- system resource-allocation graphs for
 - describing, 321-323
- Debian**, 779
- debuggers**, 66
- debugging**, 95-100, 165
- dedicated devices**, 502
- deduplication**, 757
- default access rights**, 680
- default heap**, 893
- default signal handlers**, 189
- defense in depth**, 653, 669
- deferred cancellation**, 190
- deferred procedure calls (DPCs)**, 841, 847
- degree of multiprogramming**, 112
- delayed revocation**, 682
- delayed-write policy**, 766
- deleting files**, 532, 542, 551
- demand paging**, 392-399, 430-436
 - basic mechanism, 393-396
 - defined, 393
 - free-frame list, 396-397
 - with inverted page tables, 433
 - and I/O interlock, 434-436
 - and page size, 431-432
 - and performance, 397-399
 - and prepaging, 430-431
 - and program structure, 433-434
 - pure, 395
 - and TLB reach, 432-433
- demand-zero memory**, 799
- demilitarized zone (DMZ)**, 659
- denial-of-service (DOS) attacks**, 622, 636
- dentry objects**, 605, 804, 805
- DEP (Data Execution Prevention)**, 827
- DES (data-encryption standard)**, 639
- design of operating systems**, 79-80
 - distributed systems, 753-757
 - Linux, 780-783
 - Windows 10, 826-838
 - application compatibility, 830-831
 - dynamic device support, 837-838
 - energy efficiency, 836-837
 - extensibility, 833-834
 - international support, 835-836
 - performance, 831-833
 - portability, 838-839
 - reliability, 828-829
 - security, 826-828
- desktop**, 59
- Desktop Activity Moderator (DAM)**, 837
- Desktop Window Manager (DWM)**, 825
- detection-algorithm usage (deadlock)**, 340-341
- deterministic modeling**, 245-247
- development kernels (Linux)**, 777
- device controllers**, 456. See also I/O
- device directory**, see **directory(-ies)**
- device drivers**, 7, 490, 785
- Device Guard**, 828
- device-management system calls**, 71-72
- device objects**, 863
- device reservation**, 511
- device stacks**, 862
- device-status table**, 508-509
- DFSs**, see **distributed file systems**
- digital certificates**, 644
- digital signatures**, 643, 828
- digital-signature algorithm**, 643
- dining-philosophers problem**, 293-295
- dir command**, 751
- direct access (files)**, 539-541
- direct blocks**, 576
- direct communication**, 128
- Direct-Compute**, 825
- direct I/O**, 504
- direct memory access (DMA)**, 15, 498-500
- direct-memory-access (DMA) controller**, 498
- directory(-ies)**, 541-550
 - active, 607, 884
 - acyclic-graph, 547-549
 - current, 546
 - fast sizing of, 592
 - file-system interface, 541-550
 - general graph, 549-550
 - implementation of, 568-570
 - lightweight directory-access protocol, 607
 - listing, 542
 - master file, 543
 - page, 381, 853
 - protecting, 554
 - root, 877
 - single-level, 542-543
 - tree-structured, 545-547
 - two-level, 543-545
 - user file, 543
- directory object**, 850
- direct virtual memory access (DVMA)**, 500
- dirty bits (modify bits)**, 402
- discretionary access control (DAC)**, 684
- disinfection, program**, 658
- disk(s)**. See also **mass-storage structure**; **RAID (redundant arrays of inexpensive disks)**
 - boot (system), 465
 - mini-, 704

- raw, 413, 464, 601
- disk arm**, 450
- disk blocks**, 40
- disk image**, 723-724
- diskless clients**, 762
- Disk Management tool**, 465
- disk-scheduling algorithms**, 460-461
- dispatched process**, 112
- dispatchers**, 203-204, 239, 840-841
- dispatcher database**, 842
- dispatcher objects**, 297, 845-846
- dispatching interrupts**, 9, 494
- dispatch latency**, 203, 228, 229
- dispatch queue**, 185
- distinguished name**, 647
- Distributed Denial-of-Service (DDoS)**
 - attacks, 636
- distributed file systems (DFSs)**, 605, 757-768
 - client-server model, 758-759
 - cluster-based model, 759-761
 - defined, 757
 - implementation techniques, 763-764
 - naming in, 761-764
 - remote file access in, 764-767
 - trends in, 767-768
 - Windows 10, 883
- distributed information systems**
 - (distributed naming services), 607
- distributed lock manager (DLM)**, 21
- distributed operating systems**, 749-753
- distributed systems**, 35-36
 - advantages of, 733-735
 - defined, 733
 - design issues in, 753-757
 - distributed file systems, 757-768
 - client-server model, 758-759
 - cluster-based model, 759-761
 - defined, 757
 - implementation techniques, 763-764
 - naming in, 761-764
 - remote file access in, 764-767
 - trends in, 767-768
 - distributed operating systems, 749-753
- distributions (GNU/Linux)**, 48
- DLLs (dynamically linked libraries)**, 76, 355-356
- DLM (distributed lock manager)**, 21
- DMA (direct memory access)**, 15, 498-500
- DMA-acknowledge**, 499
- DMA controller**, 498
- DMA-request**, 499
- DMZ (demilitarized zone)**, 659
- DNS (domain-name system)**, 607, 739-740
- dockers**, 719
- document(s)**:
 - File System Hierarchy Standard, 778-779
 - living, 653
- domains**:
 - capability lists for, 680-681
 - protection, 671-675, 711
 - public domain software, 779-780
 - scheduling, 238
 - security, 659
 - Windows 10, 884
- domain-name system (DNS)**, 607, 739-740
- domain switching**, 673, 674
- DOS attacks**, 622, 636
- double buffering**, 499, 509
- double caching**, 584
- double indirect blocks**, 576
- doubly linked lists**, 37
- down time**, 572
- DPCs (deferred procedure calls)**, 841, 847
- DRAM (dynamic random-access memory)**, 11
- drive formatting**, 463, 464
- drive mirroring**, 476
- driver end (STREAM)**, 519
- driver objects**, 863
- driver-registration system (Linux)**, 784, 785
- Drive Writes Per Day (DWPD)**, 453
- dropped packets**:
 - TCP transfer with, 748, 749
 - UDP transfer with, 747-748
- dual-booted systems**, 601
- dual-core design**, 17, 18
- dual-mode operation**, 24-25
- DVMA (direct virtual memory access)**, 500
- DWM (Desktop Window Manager)**, 825
- DWPD (Drive Writes Per Day)**, 453
- dynamically linked libraries (DLLs)**, 76, 355-356
- dynamic device support (Windows 10)**, 837-838
- dynamic linking**, 803
- dynamic loading**, 355
- dynamic protection**, 673
- dynamic random-access memory (DRAM)**, 11
- dynamic storage-allocation problem**, 358, 571
- dynamic tick**, 836-837

- earliest-deadline-firsts (EDF) scheduling, 232-233
- ease of use, 4, 822
- easily remembered passwords, 651
- eBPF tracing tool, 99, 100
- ec2, 44
- ECC (error-correction code), 462-463
- economic benefits of multithreaded processes, 162
- EDF (earliest-deadline-first scheduling), 232-233
- edu (top-level domain), 739
- effective access time, 397-398
- effective capabilities, 685
- effective memory-access time, 367
- effective transfer rates, 451, 486
- effective UID, 34
- efficiency, 1, 582-583, 692, 836-837
- electrical storage systems, 14
- elevator algorithm, *see* SCAN scheduling algorithm
- ELF (Executable and Linkable Format), 76-77, 801, 802
- embedded computers, 5
- empty processes, 123
- empty slabs, 429, 798
- emulation, 34, 717-718
- emulators, 703
- encapsulation (Java), 696
- encrypted viruses, 633
- encryption, 638-645
 - asymmetric, 641, 645
 - authentication, 641-644
 - defined, 638
 - key distribution, 644-645
 - public-key, 641
 - symmetric, 639-640
- energy efficiency, 836-837
- enhanced second-chance
 - page-replacement algorithm, 410-411
- entitlements (Darwin), 686-687
- entry section, 260
- entry set, 303, 304, 307
- environment:
 - computing, 40-46
 - kernel, 88
 - operating system as, 4
 - programming, 703, 717
 - run-time, 64-65
 - thread environment blocks, 889-890
- environment vector, 787
- equal allocation, 414
- errors, 462-463, 467, 511-512
- error-correcting organization, 476-477
- error-correction code (ECC), 462-463
- error detection, 57, 462
- error handling, 511-512
- eSATA buses, 456
- escalate privileges, 34
- escape (operating systems), 503
- Ethernet packets, 745-746
- events, 297
- event latency, 227-228
- event objects (Windows 10), 845
- event tracing, 822
- event-vector table, 11
- eVM, 729
- "exactly once" functionality, 150
- exceptions, 22, 497, 847-848
- exception dispatcher, 847
- exclusive locks, 534
- exec() system call, 188, 786-789
- Executable and Linkable Format, *see* ELF
- executable files 75, 107, 530
- executing files, 551
- execution of user programs, 801-803
- execution time, 353
- executive (Windows 10), 848-874
 - booting, 872-874
 - cache manager, 864-866
 - facilities for client-server computing, 861-862
 - I/O manager, 862-864
 - object manager, 849-851
 - plug-and-play manager, 869-870
 - power manager, 870-871
 - process manager, 858-860
 - registry, 871-872
 - security reference monitor, 866-869
 - virtual memory manager, 851-858
- exit section, 260
- exit() system call, 121-122
- expansion bus, 490
- exponential average, 208
- export list, 612
- ext2 (second extended file system), 805
- ext3 (third extended file system), 805-807
- ext4 (fourth extended file system), 805
- extended file attributes, 531
- extended file system (extfs), 566, 805
- extensibility, of Windows 10, 833-834
- extent (contiguous space), 572
- external data representation (XDR), 150
- external fragmentation, 359-360, 571-572
- extfs (extended file system), 566, 805

- failure(s), 473, 474, 754-756
- failure analysis, 95-96
- failure modes (remote file systems), 607-608
- fairness parameter, 307
- fair scheduling, 791
- false negatives, 656
- false positives, 656
- fast directory sizing, 592
- fast I/O mechanism, 865-866
- fast-user switching, 825, 874-875
- FAT (file-allocation table), 574-575
- fault, page, 394-395
- fault tolerance, 19, 754
- fault-tolerant systems, 754
- FC (fiber channel), 470
- FCB (file-control block), 565, 567
- FC buses, 456
- FC bus controller, 491
- FCFS scheduling, 458, 459
- FCFS scheduling algorithm, 206-207, 458
- fd (file descriptor), 568, 788
- fences, memory, 266
- fibers 241, 889-890
- fiber optic cables, 736
- fiber channel (FC), 470
- fiber channel (FC) buses, 456
- fiber channel (FC) bus controller, 491
- fidelit , 704
- FIFO, 38
- FIFO page replacement algorithm, 404-406
- 50-percent rule, 359
- file(s) 29-30, 529-530. See also **directory(-ies)**; specific types
 - appending, 551
 - attributes of, 530-531
 - defined, 255, 527, 529
 - deleting, 532, 542, 551
 - executing, 551
 - internal structure of, 537-539
 - locking, 534-536
 - opening, 532
 - operations on, 532-536
 - paging, 851
 - reading, 532, 551
 - renaming, 542
 - searches for, 541, 542
 - truncating, 532
 - writing, 532, 551
- file-allocation table (FAT), 574-575
- file-control block (FCB), 565, 567
- file descriptor (fd), 568, 788
- file handle, 568
- file info window (macOS), 531
- file management, 74
- file-management system calls, 71
- file mapping, 555, 557
- file migration, 761-762
- file modification 74
- file objects, 605, 804-805, 862
- file-open count, 534
- file-organization module, 565
- file pointers, 534
- file reference, 876
- file replication, 761
- file-serve system, 43
- file sessions, 608
- file sharing, 602-603
- file systems, 597-616
 - Andrew, 759
 - Apple, 592
 - basic, 564, 565
 - clustered, 768
 - common Internet, 607, 880
 - consistency semantics in, 608-609
 - defined, 564
 - distributed, see **distributed file systems (DFSs)**
 - extended, 566, 805
 - file sharing, 602-603
 - Google, 759-761
 - Hadoop, 484
 - Linux, 803-810
 - ext3 file system, 805-807
 - journaling, 808
 - /proc file system, 808-810
 - virtual, 804-805
 - log-based transaction-oriented, 587-588
 - logical, 565
 - mounting of, 598-602
 - network, 610-615
 - network file, 610-615, 759
 - operations, 566-568
 - parallel, 768
 - partitions in, 601-602
 - registration of, 785
 - remote, 605-608
 - Solaris, 482-484, 597, 599
 - special-purpose, 597-598
 - structure, 564-566
 - traversing, 542
 - traversing of, 542
 - UNIX, 565-566, 598
 - usage, 568
 - virtual, 603-605, 804-805
 - Windows 10, 875-879
 - write-anywhere file layout, 589-593

- ZFS, 482-484, 581, 588, 598
- file-syste context**, 788
- File System Hierarchy Standard document**, 778-779
- fil system implementation**, 563-593
 - allocation methods, 570-578
 - contiguous allocation, 570-573
 - indexed allocation, 575-577
 - linked allocation, 573-575
 - performance, 577-578
 - directory implementation, 568-570
 - efficiency, 582-583
 - file-system operations, 566-568
 - file-system structure, 564-566
 - free-space management, 578-582
 - performance, 583-586
 - recovery, 586-589
 - WAFL example, 589-593
- file-syste interface**, 529-560
 - access methods, 539-541, 551-554
 - directory structure, 541-550
 - acyclic-graph directories, 547-549
 - general graph directory, 549-550
 - single-level directory, 542-543
 - tree-structured directories, 545-547
 - two-level directory, 543-545
 - file attributes, 530-531
 - and file concept, 529-530
 - file operations, 532-536
 - file structure, 537-539
 - file types, 536-537
 - memory-mapped files, 555-560
 - protection, 550-555
- file-syste management**, 29-30
- file-syste manipulation (operating system service)**, 56-57
- fil table**, 788
- fil transfer**, 750-751
- fil viruses**, 632
- filte drivers**, 863
- filtering system-call**, 688
- filte management**, 863
- fine-graine multithreading**, 222
- Finish (data structure)**, 335, 339
- fire alls**, 41, 659-660
- fire all chains**, 815
- fire all management**, 815
- firm are**, 11, 12
- first-come first-serve (FCFS) scheduling algorithm**, 206-207, 458
- first-fit strategy**, 358, 359
- first-leve interrupt handler (FLIH)**, 496
- firs readers**, 291
- flas translation layer (FTL)**, 453-454
- flexibilit , of compiler-based enforcement**, 692
- FLIH (first-leve interrupt handler)**, 496
- flo control**, 519, 748, 749
- flushing**, 366
- folder redirection**, 883
- folders**, 59
- foreground priority separation boost**, 843
- foreground processes**, 115, 122, 215, 241
- fork() and exec() process model (Linux)**, 786-789
- fork-join model**, 180-184
- fork()system call**, 118-119, 188, 786-789
- formatting**, 463, 464
- forwarding**, 466
- forward-mapped page tables**, 371
- 4-byte pages**, 363, 364
- four-layered model of security**, 623-625
- fourth extended file system (ext4)**, 805
- fragments, packet**, 815
- fragmentation**, 359-360, 571-572
- frame(s)**, 360
 - in data-link layer, 742
 - free, 364, 396-397, 425-426
 - minimum number of, 413-414
 - page, 853
 - page faults vs., 404, 405
 - victim, 402
- frame allocation**, 413-419
 - allocation algorithms, 403, 414-415
 - equal, 414
 - global vs. local, 415-418
 - minimum number of frames, 413-414
 - non-uniform memory access, 418-419
 - proportional, 414-415
- frame-allocation algorithm**, 403, 414-415
- frame table**, 365
- free-behind technique**, 585
- FreeBSD**, 70, 71
- free frames, allocation and**, 364
- free-frame list**, 396-397, 425-426
- free page**, 856
- Free Software Foundation (FSF)**, 48
- free-space list**, 578-579
- free-space management**, 578-582
- fresh value**, 647
- front-end processors**, 522
- FSF (Free Software Foundation)**, 48
- fsgid property**, 818
- fsuid property**, 818
- FTL (flas translation layer)**, 453-454
- full backup**, 589
- full slabs**, 429, 798

functional programming languages,
313-314

G

Galois field math, 478
Gantt chart, 206
garbage collection, 454, 549, 550, 727
GB (gigabyte), 11
gcc (GNU C compiler), 778
GCD (Grand Central Dispatch), 185-186
GDT (global descriptor table), 379
general graph directories, 549-550
general revocation, 682
gestures, 60
get command, 751
GFS (Google file system), 759-761
gigabyte (GB), 11
git, 50
global allocation, 415-418
global descriptor table (GDT), 379
global dispatch queues, 185
global replacement, 415-418
global table, 679
GNOME desktop, 60
GNU C compiler (gcc), 778
GNU General Public License (GPL), 48
GNU/Linux, 48
Google Android, 42
Google file system (GFS), 759-761
GPFS, 768
GPL (GNU General Public License), 48
GPU (graphics processing unit), 735
graceful degradation, 19
Grand Central Dispatch (GCD), 185-186
granularity, minimum, 791
graphs, acyclic, 547
graphical user interfaces (GUIs), 56, 59-61
graphics processing unit (GPU), 735
green threads, 167
group (user class), 551
group identifiers 33-34
grouping, 580
group policies, 884
group rights (Linux), 817
GRUB, 94
guard pages, 852
guest (operating system), 34
guest processes, 702
GUIs (graphical user interfaces), 56, 59-61

H

hackers, 622

Hadoop, 22
Hadoop distributed file system (HDFS),
759, 760
Hadoop file system (HDFS), 484
HAL (hardware-abstraction layer), 835,
840
handles, 849
handle tables, 849
handling, signal, 188-189
handshake, three-way, 748
hard affinity, 225
hard-coding techniques, 129
hard disk drives (HDDs), 13
 components of, 450-451
 defined, 449
 scheduling, 457-461
hard errors, 463, 467
hard limits, 438, 857
hard links, 532, 549, 879
hard page faults, 416
hard real-time systems, 227
hardware, 4
 instructions, 266-269
 I/O system, 490-500
 direct memory access, 498-500
 interrupts, 494-498
 polling, 493-494
 and main memory, 350-352
 and memory management, 28
 process migration and, 753
 for relocation and limit registers, 357
 for storing page tables, 365-368
 synchronization, 265-270
 transformation of requests to operations
 by, 516-519
 virtual machine, 710-713
hardware-abstraction layer (HAL), 835,
840
hardware objects, 672
hardware threads, 222
hardware transactional memory (HTM),
312
hard working-set limits, 438
hash collision, 39
hashed page tables, 373-374
hash functions, 38-39, 643
hash map, 39
hash tables, 570
hash value (message digest), 643
HBA (host bus adapter), 491
HDDs, see hard disk drives
HDFS (Hadoop distributed file system),
759, 760
HDFS (Hadoop file system), 484

head crash, 451
 heaps, 893-894
 heap section (of process), 107
 heartbeat procedure, 754-755
 heterogeneous multiprocessing, 226-227
 hibernation, 870
 hierarchical paging, 371-373
 high-availability service, 19
 high contention, 271, 286
 high memory, 795
 high-performance computing, 20
 high-performance event timer (HPET), 505
 high priority, 212
 hijacking, session, 623
 hit ratio, 367, 432
 hives, 871
 hold-and-wait condition (deadlocks), 321, 327-328
 holes, 358
 HoloLens, 874
 honeypot, 655-656
 horizontal scalability, 484
 host(s):
 distributed system, 734
 operating system, 35, 177
 virtual machine, 702
 host-attached storage, 470
 host bus adapter (HBA), 491
 host controller, 456
 host-id, 739
 host name, 73, 738
 hot spare (drive), 480
 hot-standby mode, 19, 20
 HPET (high-performance event timer), 505
 HTM (hardware transactional memory), 312
 HTTP protocol, 881
 huge pages, 363, 432
 Hybrid Boot, 873-874
 hybrid cloud, 44
 hybrid operating systems, 86-91
 hypercalls, 717
 hypercall interface, 839
 hyper-threading, 222, 832
 Hyper-V for Client, 831
 hyper-V hypervisor, 839
 hypervisors, 670, 702
 separation, 729
 type 0, 702, 713-714
 type 1, 703, 714-715
 type 2, 703, 715-716

I

IA-32 architecture, 379-382
 IA-64 architecture, 382
 IaaS (infrastructure as a service), 44
 IB (InfiniBand) 473
 icons, 59
 ideal processors, 242, 842
 idempotent, 759
 identifiers
 address-space, 366
 file, 530
 group, 33-34
 host names vs., 738
 location-independent file, 764
 process, 116
 spoofed, 606
 user, 33
 idle process, 872
 idle threads, 239, 842
 IKE protocol, 646
 image, disk, 723-724
 immediate revocation, 682
 immutable shared files, 609
 immutable-shared-fil semantics, 609
 imperative languages, 313
 impersonation, 867
 implementation:
 CPU scheduling algorithm, 249
 cryptography, 645-646
 directory, 568-570
 file system, see **fil system**
 implementation
 monitor, 280-281
 of naming techniques, 763-764
 of operating systems, 80-81
 Pthread, 169, 170
 of security defenses, 653-662
 and accounting, 659
 and auditing, 659
 and firewalling, 659-660
 and intrusion prevention, 655-657
 levels of defenses, 661-662
 and logging, 659
 and security policy, 653
 and virus protection, 657-659
 and vulnerability assessment, 653-655
 semaphore, 274-276
 synchronization primitive, 845-846
 virtual machine, 713-719
 application containment, 718-719
 emulation, 717-718
 paravirtualization, 716-717

- programming-environment virtualization, 717
 - type 0 hypervisors, 713-714
 - type 1 hypervisors, 714-715
 - type 2 hypervisors, 715-716
 - and virtual machine life cycle, 713
- implicit threading, 176-188**
 - fork join, 180-183
 - Grand Central Dispatch, 185-186
 - Intel thread building blocks, 186-188
 - OpenMP and, 183-185
 - thread pools and, 177-180
- importance hierarchy (Android), 122**
- include file 40**
- increase scheduling priority privilege, 887**
- incremental backup, 589**
- indefinit blocking (starvation), 213, 343**
- independence, location, 762, 769**
- independent processes, 123**
- indexes, 540, 542, 576**
- index blocks, 575-577**
- indexed allocation, 575-577**
- index root, 876**
- indirect blocks, 576, 577**
- indirect communication, 129**
- indirection, 683, 703**
- InfiniBand (IB), 473**
- information leak attacks, 827**
- information-maintenance system calls, 72**
- information sharing, 123**
- infrastructure as a service (IaaS), 44**
- inheritable capabilities, 685**
- init process, 117**
- in-memory file-system structures, 568, 569**
- inode, 482, 565, 577**
- inode objects, 605, 804**
- input/output, see I/O**
- input/output operations per second (IOPS), 461**
- insert() method, 305-307**
- InServ storage array, 481**
- instruction register, 12**
- integrity, 622, 687-688**
- integrity label (Windows 10), 663**
- integrity levels, 826**
- Intel processors, 379-382**
 - event-vector table, 496
 - IA-32 architecture, 379-382
 - IA-64 architecture, 382
 - thread building blocks, 186-188
- inter-computer communications, 522**
- interface(s). See also specific types**
 - choice of, 60-62
 - defined, 501
 - speeds of, 510
- interlock, I/O, 434-436**
- intermachine interface, 757**
- internal fragmentation, 359**
- international support (Windows 10), 835-836**
- Internet, 737**
- Internet Key Exchange (IKE), 646**
- Internet model, 742**
- Internet Protocol (IP), 743-746. See also Transmission Control Protocol/Internet Protocol (TCP/IP)**
- Internet Service Providers (ISPs), 737**
- interpreted languages, 717**
- interprocess communication (IPC), 123-153**
 - in client-server systems, 145-153
 - remote procedure calls, 149-153
 - sockets, 146-149
 - in Linux, 777, 812-813
 - Mach example of, 135-138
 - in message-passing systems, 127-132
 - pipes in, 139-145
 - POSIX shared-memory example of, 132-135
 - in shared-memory systems, 125-127
 - Windows example of, 138-139
- interrupt(s), 8-11, 494-498**
 - defined, 494
 - in Linux, 794
 - maskable, 10, 495-496
 - nonmaskable, 10, 495
 - software (traps), 497
 - in Windows 10, 846-848
- interrupt chaining, 10, 496**
- interrupt-controller hardware, 9, 495**
- interrupt-dispatch table (Windows 10), 848**
- interrupt-handler routine, 9, 494-495**
- interrupt latency, 228**
- interrupt objects, 848**
- interrupt priority levels, 10-11, 497**
- interrupt request levels (IRQs), 841, 846**
- interrupt-request line, 9, 494**
- interrupt service routines (ISRs), 844**
- interrupt vector, 9, 496**
- intruders, 622**
- intrusion prevention, 655-657**
- intrusion-prevention systems (IPSs), 656**
- inverted page tables, 374-375, 433**
- involuntary context switches, 204**
- I/O (input/output):**
 - fast mechanism for, 865-866

- raw, 464, 465
- structure of, 14-15
- in virtual machines, 722-723
- I/O-bound processes, 112**
- I/O burst, 201**
- I/O bus, 456**
- I/O channel, 522**
- I/O control level (fil system), 564**
- I/O interlock, 434-436**
- I/O manager, 862-864**
- I/O operations, 56**
- IOPS (input/output operations per second), 461**
- I/O request packet (IRP), 863**
- iOS operating system, 42, 87-89**
- I/O status information (PCBs), 110**
- I/O subsystem(s), 32-33**
 - kernels in, 508-516
 - procedures supervised by, 516
- I/O system(s), 489-525**
 - application interface, 500-508
 - block and character devices, 503-504
 - clocks and timers, 505-506
 - network devices, 504-505
 - nonblocking and asynchronous I/O, 506-507
 - vectored I/O, 507-508
 - hardware, 490-500
 - direct memory access, 498-500
 - interrupts, 494-498
 - for memory-mapped I/O, 491-493
 - polling, 493-494
 - summary, 500
 - kernel subsystem, 508-516
 - buffering, 509-510
 - caching, 510-511
 - data structures, 512-514
 - error handling, 511-512
 - I/O scheduling, 508-509
 - power management, 514-516
 - procedures supervised by, 516
 - protection, 512
 - spooling and device reservation, 511
 - Linux, 810-812
 - overview, 489-490
 - STREAMS mechanism, 519-521
 - and system performance, 521-524
 - transformation of requests to hardware operations, 516-519
- IP (Internet Protocol), 743-746.** See also **Transmission Control Protocol/Internet Protocol (TCP/IP)**
- IPC, see interprocess communication**
- iPhone, 60**

- IPIs (Windows 10), 847-848**
- IPSec, 646**
- IPSs (intrusion-prevention systems), 656**
- IRP (I/O request packet), 863**
- IRQLs (interrupt request levels), 841, 846**
- iSCSI, 471**
- ISPs (Internet Service Providers), 737**
- ISRs (interrupt service routines), 844**
- Itanium, 382**
- iteration space, 187**

J

Java:

- DNS lookup in, 740
- file locking in, 534, 535
- fork-join in, 180-184
- Lambda expressions in, 174
- language-based protection in, 694-696
- synchronization, 303-311
 - condition variables, 309-311
 - monitors, 303-307
 - reentrant locks, 307-308
 - semaphores, 308-309
 - thread dumps in, 339
 - thread pools in, 179-180
- Java Executor interface, 175-176**
- Java threads, 173-176**
- Java Virtual Machine (JVM), 177, 717, 727-728**
- JBOD (Just a Bunch of Disks), 472**
- JIT compilers, 728**
- jobs, processes vs., 106**
- job objects, 859**
- job scheduling, 106**
- journaling, 587-588, 808**
- Just a Bunch of Disks (JBOD), 472**
- just-in-time (JIT) compilers, 728**
- JVM, see Java Virtual Machine**

K

- KB (kilobyte), 11**
- K Desktop Environment (KDE), 60**
- Kerberos network authentication protocol, 607**
- kernel(s), 6, 7, 501, 508-516**
 - buffering, 509-510
 - caching, 510-511
 - computational, 833
 - data structures, 36-40, 512-514
 - error handling, 511-512
 - I/O scheduling, 508-509
 - and I/O subsystems, 516

- Linux, 776-778, 781
 - nonpreemptive, 262
 - power management, 514-516
 - preemptive, 262
 - protection, 512
 - secure, 839-840
 - spooling and device reservation, 511
 - synchronization of, 295-299, 792-794
 - uni-, 728
 - Windows 10, 839-848
 - kernel abstractions, 89**
 - kernel code, 261**
 - kernel data structures, 36-40, 512-514**
 - kernel environment, 88**
 - kernel extensions (kexts), 89**
 - kernel memory allocation, 426-430**
 - kernel mode, 24, 25, 782**
 - Kernel-Mode Driver Framework (KMDF), 864**
 - kernel-mode threads (KT), 841**
 - kernel modules, 86, 783-786**
 - kernel module management, 784**
 - kernel object access (Windows 10), 884-885**
 - kernel threads, 166, 217, 234**
 - kernel virtual memory, 801**
 - Kernighan's Law, 98**
 - kexts (kernel extensions), 89**
 - keys:**
 - for capabilities, 683
 - defined, 638
 - Internet Key Exchange, 646
 - in lock-key schemes, 681
 - master, 683
 - private, 641
 - public, 641
 - sense, 512
 - session, 647
 - key distribution, 644-645**
 - key ring, 644**
 - keystreams, 640**
 - keystroke logger, 634**
 - kilobyte (KB), 11**
 - KMDF (Kernel-Mode Driver Framework), 864**
 - Korn shell, 58**
 - KT (kernel-mode threads), 841**
 - Kubernetes, 719**
- L**
- labels, for mandatory access control, 685**
 - Lambda expressions, 174**
 - languages, 313-314, 717**
 - language-based protection systems, 690-696**
 - compiler-based enforcement, 691-693
 - in Java, 694-696
 - LANs (local-area networks), 36, 735-737**
 - large objects, 430**
 - latency:**
 - dispatch, 203, 228, 229
 - event, 227-228
 - interrupt, 228
 - in real-time systems, 227-229
 - rotational, 451
 - target, 236, 791
 - latency command, 494**
 - layers (of network protocols), 645**
 - layered approach (operating system structure), 83-84**
 - layered protocols, 891**
 - LBA (logical block address), 456**
 - LCNs (logical cluster numbers), 875**
 - LDAP (lightweight directory-access protocol), 607, 884**
 - LDT (local descriptor table), 379**
 - least-frequently used (LFU)**
 - page-replacement algorithm, 411-412
 - least privilege, principle of, 626, 627, 668-669**
 - least-recently-used (LRU) algorithm, 407-408**
 - left child, 38**
 - LFH design, 894**
 - LFU page-replacement algorithm, 411-412**
 - lggroups, 419**
 - libc (C library), 63, 69, 370, 781**
 - libraries:**
 - C, 63, 69, 370, 781
 - Linux system, 781
 - shared, 356, 392
 - thread, 168-176
 - about, 168-169
 - Java, 173-176
 - Pthreads, 169-171
 - Windows, 171-173
 - library operating systems, 728**
 - licensing, Linux, 779-780**
 - life cycle:**
 - I/O request, 518-519
 - virtual machine, 713
 - lifetime, virtual address space, 799-800**
 - LIFO, 37-38**
 - lightweight directory-access protocol (LDAP), 607, 884**
 - lightweight process (LWP), 193**

- limit register, 351-352**
- linear addresses, 380, 382**
- linear lists (files) 569-570**
- line discipline, 811-812**
- link(s):**
 - communication, 128
 - defined, 548
 - hard, 532, 549, 879
 - resolving, 548
 - symbolic, 879
- linked allocation, 573-575**
- linked lists, 37, 38, 579-580**
- linked scheme index block, 576**
- linkers, 75, 76**
- linking, 355-356, 803, 882**
- Linux, 48, 775-819**
 - capabilities in, 685-686
 - design principles for, 780-783
 - file systems, 803-810
 - ext3 file system, 805-807
 - journaling, 808
 - /proc file system, 808-810
 - virtual, 804-805
 - history of, 775-780
 - input and output, 810-812
 - interprocess communication, 812-813
 - kernel modules, 783-786
 - lockdep tool, 330
 - memory management, 795-803
 - execution and loading of user programs, 801-803
 - physical memory, 795-798
 - virtual memory, 436-437, 798-801
 - network structure, 813-815
 - process management, 786-790
 - process representation in, 111
 - scheduling in, 234-239, 790-794
 - security model, 816-818
 - swap-space management in, 468-470
 - synchronization in, 296-298
 - system structure, 83
 - threads example, 195-196
 - tree of processes, 116
 - Windows subsystem for, 91
- Linux distributions, 776, 779**
- Linux instance, 91**
- Linux kernel, 776-778, 781**
- Linux kernel data structures, 40**
- Linux system(s), 776**
 - components of, 781-783
 - history of, 778-779
 - obtaining page size on, 364
- Linux timers, 27**
- lists, 37**
 - access, 679-680
 - access-control, 552, 555, 826
 - capability, 680-681
 - export, 612
 - free-frame, 396-397, 425-426
 - free-space, 578-579
 - linear, 569-570
 - linked, 37, 38, 579-580
 - user control, 561
- listing directories, 542**
- listing file names and attributes, 551**
- little cores, 227**
- little-endian, 150**
- Little's formula, 247**
- live CD, 48**
- live DVD, 48**
- livelock, 320-322**
- live migration (virtual machines), 706, 724-726**
- liveness, 283-284**
- living documents, 653**
- loadable kernel modules (LKMs), 86**
- load balancing, 224-225, 735, 753**
- loaders, 75-77, 695, 727, 783. See also bootstrap programs**
- loading, 355, 801-803**
- load sharing, 220**
- load time, 353**
- local allocation, 415-418**
- local-area networks (LANs), 36, 735-737**
- local descriptor table (LDT), 379**
- locality model, 421**
- locality of reference, 395**
- locality property, 857**
- local-name, 763**
- local replacement, 415-418**
- local replacement algorithm, 420-421**
- location, file 530, 534**
- location independence, 761, 762**
- location-independent file identifiers 764**
- location transparency, 761**
- locks, 681. See also deadlock(s)**
 - advisory, 535
 - exclusive, 534
 - in Java API, 534, 535
 - mandatory, 535
 - mutex, 270-272, 299-300
 - nonrecursive, 299
 - Pushlocks, 831
 - reader-writer, 292-293
 - reentrant, 307-308
 - scope of, 305
 - shared, 534
 - for shared data, 70

- lock-free algorithms, 284
- locking, page, 434-436
- locking files, 534-536
- lock-key scheme, 681
- lofs (fil system), 598
- log-based transaction-oriented fil systems, 587-588
- log files, 95, 876
- log-fil service, 878
- logging, 57, 659
- logging area, 878
- logical address, 353, 379
- logical address space, 353-355
- logical blocks, 456
- logical block address (LBA), 456
- logical cluster numbers (LCNs), 875
- logical file system, 565
- logical formatting, 464
- logical memory, 24, 362. See also **virtual memory**
- logical processors, 832
- logical records, 539
- logic bomb, 627
- login, remote, 750
- loopback, 148
- loosely coupled system, 83
- loosely-coupled systems, 19
- love bug virus, 658
- low contention, 271
- low-fragmentation heap (LFH) design, 894
- low-level formatting (disks), 463
- low priority, 212
- LRU algorithm, 407-408
- LRU-approximation page replacement algorithm, 409-411
- LRU page replacement, 407-409
- ls command, 751
- Lustre, 768
- LWP (lightweight process), 193
- LXC containers, 718, 719

M

- MAC (mandatory access control), 684-685
- MAC (message-authentication code), 643
- MAC address, 745
- Mach-O format, 77
- Mach operating system, 84, 135-138
- macOS operating system:
 - GUI, 61
 - as hybrid system, 87-89
 - latency command, 494
 - sandboxing in, 690-691

- macro viruses, 632
- magic number (files) 537
- magnetic tapes, 455
- mailboxes, 129-130
- main memory, 349-385
 - and address binding, 352-353
 - ARMv8 architecture, 383-384
 - contiguous allocation of, 356-360
 - and dynamic linking, 355-356
 - and dynamic loading, 355
 - and hardware, 350-352
 - Intel 32 and 64-bit architectures, 379-382
 - and logical vs. physical address space, 353-355
 - paging for management of, 360-376
 - basic method, 360-365
 - hardware, 365-368
 - hashed page tables, 373-374
 - hierarchical paging, 371-373
 - inverted page tables, 374-375
 - and Oracle SPARC Solaris, 375-376
 - protection, 368-369
 - and shared pages, 369-371
 - swapping with, 377
 - shared libraries, 356
 - and swapping, 376-378
- main queue, 185
- main TLB, 384
- major page faults, 416
- malware, 625-628
- MANs (metropolitan-area networks), 36
- mandatory access control (MAC), 684-685
- mandatory file-lockin mechanisms, 535
- mandatory policy, 826
- man-in-the-middle attack, 623, 635, 645
- many-to-many multithreading model, 167-168
- many-to-one multithreading model, 166-167
- mapping, 39
 - address, 456-457
 - file, 555, 557
 - memory, 802-803
- MapReduce system, 22, 761
- marshaling, 150, 882
- Mars Pathfinder, 285
- maskable interrupts, 10, 495-496
- masquerading, 622, 623, 635
- mass-storage management, 30
- mass-storage structure, 449-486
 - address mapping, 456-457
 - attachment of storage, 469-473
 - device management, 463-467
 - error detection and correction, 462-463

- hard disk drives, 450-451, 457-461
- nonvolatile memory devices, 452-454, 461-462
- overview, 449-450
- RAID, 473-485
 - extensions, 481-482
 - for object storage, 483-485
 - performance improvement, 475
 - problems with, 482-483
 - RAID levels, 475-481
 - reliability improvement, 473-475
- scheduling, 457-462
- secondary storage connection methods, 456
- swap-space management, 467-469
- volatile memory, 454-455
- master book record (MBR)**, 465, 466
- master file directory (MFD)**, 543
- master file table**, 566
- master key**, 683
- master secret (TLS)**, 647
- matchmakers**, 151
- Max (data structure)**, 335
- maximum number of resources**,
 - declaration of, 330
- MB (megabyte)**, 11
- MBR (master book record)**, 465, 466
- MD5 message digest**, 643
- mean time between failures (MTBF)**, 473, 474
- mean time of data loss**, 474
- mean time to repair**, 474
- mechanical storage systems**, 14
- mechanisms**, 80, 668. See also specific mechanisms
- medium access control (MAC) address**, 745
- medium objects**, 430
- megabyte (MB)**, 11
- memory**:
 - Address Window Extension, 894-895
 - anonymous, 399, 469
 - defined, 14
 - demand-zero, 799
 - direct memory access, 14, 498-500
 - direct virtual memory access, 500
 - high, 795
 - in-memory file-system structures, 568, 569
 - layout of, in C program, 108
 - layout of process in, 106
 - logical, 24, 362
 - main, see **main memory**
 - network virtual memory, 765
 - over-allocation of, 401
 - physical, 24, 362, 390, 391, 795-798
 - secondary, 395
 - semiconductor, 14
 - shared, 57, 73, 123, 125, 556-560
 - software transactional, 312
 - 32-byte, 363, 364
 - transactional, 311-312
 - virtual, see **virtual memory**
 - volatile, 454-455
- memory access**:
 - direct, 15, 498-500
 - direct virtual, 500
 - effective memory-access time, 367
 - non-uniform, 18, 19, 418-419
- memory-address register**, 354
- memory allocation**, 358-359, 426-430
- memory barriers**, 265-266
- memory compression**, 425-426, 858
- memory devices**:
 - management of, 463-467
 - nonvolatile, 452-454
 - defined, 449
 - NAND flash controller algorithms for, 453-454
 - overview, 452-453
 - scheduling, 461-462
- memory fences**, 266
- memory management**, 28-29
 - in Linux, 795-803
 - execution and loading of user programs, 801-803
 - physical memory, 795-798
 - virtual memory, 798-801
 - with virtual machines, 721-722
 - in Windows 10, 892-895
- memory-management information (PCBs)**, 110
- memory-management unit (MMU)**, 354, 855
- memory manager (MM)**, 851
- memory-mapped files**, 555-560, 892-893
- memory-mapped I/O**, 491-493
- memory mapping (Linux)**, 802-803
- memory model**, 265-266
- memory protection**, 357, 368-369
- memory stall**, 221-222
- memory-style error-correcting organization**, 476-477
- memory transactions**, 311
- messages**, 135
 - complex, 136
 - in distributed systems, 734
 - OSI network, 742, 744

- in Win32 API, 891
- message-authentication code (MAC)**, 643
- message digest (hash value)**, 643
- message modification** 622-623
- message passing**, 123, 125, 130
- message-passing model**, 57, 72-73, 127-132
 - buffering, 131-132
 - Mach example, 135-138
 - naming, 128-130
 - synchronization, 130-131
- metadata**, 607, 876
- metaslabs**, 581
- methods (Java)**, 694
- Metro**, 823
- metropolitan-area networks (MANs)**, 36
- MFD (master file directory)**, 543
- MFU page-replacement algorithm**, 412
- microkernels**, 84-86
- Microsoft Interface Definition Language (MIDL)**, 150, 882
- Microsoft Windows**, see **Windows operating system (generally)**
- micro TLBs**, 384
- middleware**, 6, 7
- MIDL (Microsoft Interface Definition Language)**, 150, 882
- migration**:
 - computation, 752
 - data, 751-752
 - file, 761-762
 - process, 752-753
 - push and pull, 224
 - with virtual machines, 706, 724-726
- minidisks**, 704
- minifilters** 863
- minimum granularity**, 791
- miniport driver**, 864
- minor page faults**, 416
- mirrored volume**, 474
- mirroring**, 474, 476
- MM (memory manager)**, 851
- MMU (memory-management unit)**, 354, 855
- mobile computing**, 41-42
- mobile systems**, 115, 377-378
- mode bits**, 24
- moderate contention**, 285
- Modern**, 823
- modified page**, 856
- modify bits (dirty bits)**, 402
- modularity**, 123
- modules**:
 - file-organization, 565
 - kernel, 86, 783-786
 - pluggable authentication, 816
 - stream, 519
- module loader**, 783
- module-management system**, 783, 784
- module unloader**, 783
- monitors**, 276-282
 - dining-philosophers solution with, 295, 296
 - implementation of, using semaphores, 280-281
 - in Java, 303-307
 - resumption of processes within, 281-282
 - security reference, 866-869
 - usage of, 277-280
- monitor calls**, see **system calls**
- monitor type**, 277
- monoculture**, 634
- monolithic operating systems**, 82-83
- Moore's Law**, 5
- most-frequently used (MFU)**
 - page-replacement algorithm, 412
- motherboard**, 20
- motivation, for multithreading**, 160-161
- mounting**, 464, 598-602
- mount points**, 598, 879
- mount protocol (NFS)**, 612
- mount table**, 517, 567
- MTBF (mean time between failures)**, 473, 474
- MUI support**, 840
- multicore processors**, 221-224
- multicore programming**, 162-166
- multicore systems**, 16-18
- multidimensional RAID level 6**, 478
- multifactor authentication**, 653
- multilevel feedback-queue scheduling algorithm**, 216-217
- multilevel index**, 576
- multilevel queue scheduling algorithm**, 214-216
- multimode operation**, 25-26
- multipartite viruses**, 634
- Multiple UNC Provider (MUP)**, 882-883
- multiple user interface (MUI) support**, 840
- multiprocessing**, 16, 220, 226-227, 794
- multiprocessors**, 18, 220
- multi-processor scheduling**, 220-227
 - approaches to, 220-221
 - examples, 234-242
 - Linux, 234-239
 - Solaris, 242-244
 - Windows, 239-242
 - heterogeneous multiprocessing, 226-227

- and load balancing, 224-225
- and multicore processors, 221-224
- and processor affinity, 225-226
- multiprocessor systems, 16-19**
- multiprogramming, 23, 112, 420**
- multi-provider router, 883**
- multitasking, 23, 115, 790**
- multithreaded processes, 160**
 - benefits of, 162
 - deadlocks in, 319-321
 - and exec() system call, 188
 - and fork() system call, 188
 - models of, 166-168
 - motivation for, 160-161
 - and signal handling, 188-189
- multithreading:**
 - chip, 222, 223
 - coarse-grained, 222
 - fine-grained, 222
 - many-to-many, 167-168
 - many-to-one, 166-167
 - one-to-one, 167
 - simultaneous, 222
- multi-touch hardware, 874**
- MUP (Multiple UNC Provider), 882-883**
- mutex locks, 270-272, 299-300**
- mutual exclusion (mutex), 260, 267, 268, 845**
- mutual-exclusion condition (deadlocks), 321, 327**

N

- names:**
 - common and distinguished, 647
 - host, 73, 738
 - resolution of, 738-741
- named condition variables, 309**
- named pipes, 143-145, 881-882**
- named semaphores, 300-301**
- named shared-memory object, 559**
- name server, 739**
- namespaces, 787**
- naming, 128-130**
 - defined, 761
 - in distributed file systems, 761-764
 - distributed naming services, 607
 - domain name system, 607
 - file, 530
 - and network communication, 738-741
 - port, 135-136
- naming schemes, 763**
- naming structures (DFS), 761-763**

- NAND flas controller algorithms, 453-454**
- NAS (network-attached storage), 470-471**
- NAT (network address translation), 723**
- national-language-support (NLS) API, 835**
- NDIS (Network Device Interface specification) 880**
- Need (data structure), 335, 336**
- need-to-know principle, 672**
- nested page tables (NPTs), 710, 712**
- network(s):**
 - communication structure, 738-749
 - communication protocols, 741-745
 - and naming/name resolution, 738-741
 - TCP/IP example, 745-746
 - UDP and TCP transport protocols, 746-749
 - defined, 36
 - firewalling to protect, 659-660
 - in Linux, 813-815
 - local-area, 36, 735-737
 - metropolitan-area, 36
 - network operating systems, 749-751
 - personal-area, 36
 - as resource types, 318
 - security in, 623
 - storage-area, 21, 470, 472
 - structure of, 735-738
 - threats to, 634-637
 - virtual private, 646, 881
 - wide-area, 36, 735, 737-738
 - wireless, 41, 736-737
- network address translation (NAT), 723**
- network-attached storage (NAS), 470-471**
- network computers, 40**
- network devices, 504-505, 810**
- Network Device Interface specification (NDIS), 880**
- network file system (NFS), 610-615, 759**
- network information service (NIS), 607**
- networking, 880-884**
- network interfaces (Windows 10), 880**
- network layer, 742**
- network-layer protocol, 645**
- network operating systems, 36, 749-751**
- network protocols, registration of, 785**
- network time protocol (NTP), 505**
- network virtual memory, 765**
- new state, 108**
- NFS (network file system), 610-615, 759**
- NFS protocol, 612-614**
- nice value, 236, 790**
- NIS (network information service), 607**

- NLS API, 835
 - no-access page, 852
 - nonblocking I/O, 506-507
 - nonblocking message passing, 130
 - noncontainer objects (Windows 10), 664
 - nonmaskable interrupts, 10, 495
 - nonpreemptive kernels, 262
 - nonpreemptive scheduling, 202
 - nonrecursive locks, 299
 - nonrepudiation, 644
 - nonresident attributes, 876
 - nonsignaled state, 297
 - non-uniform memory access (NUMA), 18, 19, 418-419
 - nonvolatile memory (NVM) devices, 13, 14, 452-454
 - defined, 449
 - NAND flash controller algorithms for, 453-454
 - overview, 452-453
 - scheduling, 461-462
 - nonvolatile storage (NVS), 14, 449
 - NOOP scheduler, 461
 - no-preemption condition (deadlocks), 321, 328
 - Normal World, 838
 - notify() method, 305-307
 - Notify port, 135
 - NPTs (nested page tables), 710, 712
 - NTFS, 875-877
 - NTP (network time protocol), 505
 - NUMA, see non-uniform memory access
 - NUMA-aware algorithms, 225-226
 - NUMA mode, 238-239
 - NVM devices, see nonvolatile memory devices
 - NVM express (NVMe), 456
 - NVS (nonvolatile storage), 14, 449
- O**
- objects:
 - access lists for, 679-680
 - in cache, 428
 - container, 664
 - critical-section, 297, 888
 - defined, 672
 - dentry, 605, 804, 805
 - device, 863
 - directory, 850
 - dispatcher, 297, 845-846
 - driver, 863
 - event, 845
 - file, 605, 804-805, 862
 - hardware, 672
 - inode, 605, 804
 - interrupt, 848
 - job, 859
 - in Linux, 797, 804
 - named shared-memory, 559
 - noncontainer, 664
 - section, 139, 852
 - semaphore, 845
 - sharing, 885-886
 - small, medium, and large, 430
 - software, 672
 - superblock, 605, 804, 805
 - timer, 845
 - in Windows 10, 664, 845-846, 848, 849
 - Object Linking and Embedding (OLE), 882
 - object manager (Windows 10), 849-851
 - object storage, 483-485
 - object types, 849, 850
 - objs (fil system), 598
 - off-line compaction of space, 572
 - OLE (Object Linking and Embedding), 882
 - one-time passwords, 652
 - one-to-one multithreading model, 167
 - on-line compaction of space, 572
 - OOM (out-of-memory) killers, 418
 - OpenAFS (Andrew fil system), 759
 - open count, 533
 - open-fil table, 533, 567
 - opening files, 532
 - OpenMP, 183-185, 312-313
 - open operating systems, 634
 - open-source operating systems, 46-51
 - Open Systems Interconnection (OSI) model, 741-744
 - operating system(s):
 - application specificity to, 77-79
 - booting, 94-95
 - building, 92-93
 - closed-source, 46
 - computing environments, 40-46
 - CPU scheduling in, 234-244
 - Linux scheduling, 234-239
 - Solaris scheduling, 242-244
 - Windows scheduling, 239-242
 - debugging, 95-100
 - defined, 1, 3, 5-7
 - design goals for, 79-80
 - features of, 3
 - functioning of, 4-7
 - implementation of, 80-81
 - kernel data structures, 36-40

- linkers and loaders, 75-77
 - network, 36
 - open-source, 46-51
 - operations, 21-27
 - dual-mode and multimode, 24-26
 - multiprogramming and multitasking, 23-24
 - and timer, 26-27
 - reasons for studying, 6
 - as resource allocator, 5
 - resource management by, 27-32
 - security in, 623-624
 - services provided by, 55-58
 - structure, 81-91
 - hybrid systems, 86-91
 - layered approach, 83-84
 - microkernels, 84-86
 - modules, 86
 - monolithic, 82-83
 - study of, 50
 - system calls, 62-74
 - and API, 63-66
 - functioning of, 62-63
 - types of calls, 66-74
 - system services, 74-75
 - system's view of, 5
 - threads in, 194-196
 - user interface with, 4-5, 58-62
 - virtualization components, 719-726
 - CPU scheduling, 720
 - I/O devices, 722-723
 - live migration, 724-726
 - memory management, 721-722
 - optimal page replacement, 406-407**
 - optimal page replacement algorithm, 406-407**
 - optimistic approach, 285**
 - Oracle SPARC Solaris, 375-376**
 - Orange Book, 826**
 - ordinary pipes, 140-143**
 - org (top-level domain), 739**
 - orphan processes, 122**
 - OS/2 operating system, 821-822**
 - OSI model, 741-744**
 - OSI network model, 741-744**
 - OSI protocol stack, 742-744**
 - other users (class), 551**
 - out-of-band key delivery, 644**
 - out-of-memory (OOM) killers, 418**
 - over-allocation of memory, 401**
 - overcommitment, 720**
 - over-provisioning, 454**
 - owners:**
 - clock, 837
 - file, 603
 - as user class, 551
 - owner rights, 678, 817**
- P**
- p (page number), 360**
 - PaaS (platform as a service), 44**
 - packaged applications, 859**
 - package systems, 823**
 - PAE (page address extension), 381**
 - pages.** See also specific types
 - defined, 360
 - locking, 434-436
 - obtaining size of, 364
 - page address extension (PAE), 381**
 - page allocator (Linux), 796**
 - page-buffering algorithms, 412**
 - page cache, 583, 798**
 - page directory, 381, 853**
 - page-directory entries (PDEs), 853**
 - page directory pointer table, 381**
 - page faults, 394-395, 405, 416**
 - page-fault-frequency (PFF), 424-425**
 - page-fault rate, 398, 423**
 - page frames, 853**
 - page-frame number (PFN) database, 856**
 - page in, 377**
 - page locking, 434-436**
 - page number (p), 360**
 - page offset (d), 360**
 - page out, 377**
 - pageout policy (Linux), 800**
 - pageout process (Solaris), 439**
 - page replacement, 401-413.** See also **frame allocation**
 - and application performance, 412-413
 - basic mechanism, 401-404
 - counting-based page replacement, 411-412
 - defined, 401
 - FIFO page replacement, 404-406
 - global vs. local, 415-418
 - LRU-approximation page replacement, 409-411
 - LRU page replacement, 407-409
 - optimal page replacement, 406-407
 - and page-buffering algorithms, 412
 - page replacement algorithm, 403**
 - page size, 363, 364, 431-432**
 - page slots, 469**
 - page table(s), 361-378, 393**
 - clustered, 374
 - defined, 361

- for demand paging, 395
 - forward-mapped, 371
 - hardware for storing, 365-368
 - hashed, 373-374
 - for hierarchical paging, 371-373
 - inverted, 374-375, 433
 - nested, 710, 712
 - Oracle SPARC Solaris, 375-376
 - page-table base register (PTBR), 365**
 - page-table entries (PTEs), 853**
 - page-table length register (PTLR), 369**
 - paging, 360-376**
 - basic method, 360-365
 - demand, 392-399, 430-436
 - basic mechanism, 393-396
 - defined, 393
 - free-frame list, 396-397
 - with inverted page tables, 433
 - and I/O interlock, 434-436
 - and page size, 431-432
 - and performance, 397-399
 - and prepaging, 430-431
 - and program structure, 433-434
 - pure, 395
 - and TLB reach, 432-433
 - hardware, 365-368
 - for hashed page tables, 373-374
 - hierarchical, 371-373
 - IA-32, 380-381
 - inverted, 374-375
 - in Linux, 800
 - and memory protection, 368-369
 - and Oracle SPARC Solaris, 375-376
 - priority, 440
 - and shared pages, 369-371
 - swapping with, 377, 378
- paging files 851**
- paired passwords, 652**
- PAM (pluggable authentication modules), 816**
- PAN (personal-area network), 36**
- parallel file system (PFS), 768**
- parallelism, 163, 165-166, 475**
- parallelization, 20**
- parallel regions, 183-184**
- paravirtualization, 703, 716-717**
- parent-child relationship, 140**
- parent process, 111**
- partial revocation, 682**
- partial slabs, 429, 798**
- partitions:**
 - boot, 465
 - control, 714
 - file-system, 601-602
 - raw, 468
 - root, 601
 - storage device, 463-465
 - variable-partition schemes, 358
- partition boot sector, 566**
- partitioning, device, 463-464**
- passphrases, 651**
- passwords, 554, 649-652**
- path names, 544, 546**
- path-name translation (NFS), 614-615**
- PB (petabyte), 12**
- PCBs (process control blocks), 109-110**
- PCIe bus, 490**
- PC motherboard, 20**
- PCS (process-contention scope), 217-218**
- PC systems, 874**
- PDEs (page-directory entries), 853**
- peer-to-peer computing, 43-44**
- peer-to-peer distributed systems, 734**
- PE (Portable Executable) format, 77**
- penetration test, 654**
- performance:**
 - and allocation of secondary storage, 578-579
 - and demand paging, 397-399
 - and file system implementation, 583-586
 - and I/O system, 521-524
 - and page replacement, 412-413
 - RAID structure to improve, 475
 - under swapping, 378
 - virtualization requirement related to, 704
 - of Windows 10, 831-833
- performance monitoring, 96-97**
- performance tuning, 95-97**
- periodic processes, 230**
- periodic task rate, 230**
- permanent revocation, 682**
- permissions, 553, 669**
- permitted capabilities, 685**
- per-process open-file table, 567**
- per-process tools, 96, 97**
- personal-area network (PAN), 36**
- personal computer (PC) systems, 874**
- personal firewalls, 660**
- personal identification number (PIN), 652**
- personalities, 87, 787**
- pessimistic approach, 285**
- petabyte (PB), 12**
- Peterson's solution, 262-265**
- PFF (page-fault-frequency), 424-425**
- PFN database, 856**
- PFS (parallel file system), 768**
- phishing, 624**

- PHY (ports), 490
- physical address, 354, 379
- physical address space, 353-355
- physical formatting, 463
- physical layer, 741, 742
- physical memory, 24, 362, 390, 391, 795-798
- physical security, 623
- physical-to-virtual (P-to-V) conversion, 724
- PIC (position-independent code), 803
- Pico process, 91
- Pico Providers, 823
- pid (process identifier) 116, 787
- PIN (personal identification number), 652
- pinning, 436, 866
- PIO (programmed I/O), 498
- pipes, 139-145
 - anonymous, 141-145
 - implementation considerations, 139-140
 - named, 143-145, 881-882
 - ordinary, 140-143
 - use of, 146
- pipe mechanism, 813
- platform as a service (PaaS), 44
- platter (disks), 450
- PLM (Process Lifetime Manager), 837
- plug-and-play and (PnP) managers, 869-870
- pluggable authentication modules (PAM), 816
- plug-in process, 124
- PnP managers, 869-870
- PoFX (power framework), 870
- Point-to-Point Tunneling Protocol (PPTP), 881
- policy(-ies), 80
 - cache updating, 766-767
 - delayed-write, 766
 - group, 884
 - mandatory, 826
 - mechanisms vs., 668
 - pageout, 800
 - security, 653
 - write-on-close, 766-767
 - write-through, 766
- policy algorithm (Linux), 800
- polling, 493-494
- polymorphic viruses, 633
- pools, 177-180, 483, 889
- pop, 66
- ports, 78, 129, 490
 - connection and communication, 138
 - naming of, 135-136
 - in remote procedure calls, 150
 - well-known, 146
- portability, 834-835
- Portable Executable (PE) format, 77
- portals, 40
- port driver, 864
- port number, 746-747
- port rights, 135
- port scanning, 637
- position-independent code (PIC), 803
- positioning time (disks), 450
- POSIX:
 - interprocess communication example, 132-135
 - real-time scheduling, 232-234
 - synchronization examples, 299-303
- POSIX 1e, 685, 686
- possession (of capability), 680
- POST (power-on self-test), 872
- posting messages, 891
- power framework (PoFX), 870
- power management, 514-516
- power manager (Windows 10), 870-871
- power-of-2 allocator, 427
- power-on self-test (POST), 872
- power users, 60-61
- PPTP (Point-to-Point Tunneling Protocol), 881
- P + Q redundancy scheme, 478
- preemptive kernels, 262
- preemptive multitasking, 790
- preemptive scheduling, 202-203
- premaster secret (TLS), 647
- prepaging, 430-431
- presentation layer, 742
- primary thread, 890
- principle of least privilege, 626, 627, 668-669
- priority (field) 243
- priority-based scheduling, 229-230
- priority-inheritance protocol, 284
- priority inversion, 284, 285
- priority number, 281
- priority paging, 440
- priority replacement algorithm, 420-421
- priority scheduling algorithm, 211-214
- private cloud, 44
- private dispatch queues, 185
- private keys, 641
- privileged instructions, 25
- privileged mode, see kernel mode
- privilege escalation, 623
- privilege separation, 669
- procedural languages, 313
- procedures, as domains, 674

process(es), 23, 105-154

- aborting, 342
- background, 74-75, 115, 123, 215, 241
- browser, 124
- communication between, see **interprocess communication**
- components of, 106-107
- consumer, 126-127, 290, 291, 559-560
- context of, 788-789
- cooperating, 123, 257
- coordination among, 260
- daemon, 690
- defined, 103, 105
- dispatched, 112
- as domains, 674
- empty, 123
- environment of, 787-788
- foreground, 115, 122, 215, 241
- guest, 702
- idle, 872
- independent, 123
- init, 117
- I/O- vs. CPU-bound, 112
- job vs., 106
- lightweight, 193
- in Linux, 789-790
- multithreaded, see **multithreaded processes; multithreading**
- operations on, 116-123
 - creation, 116-121
 - termination, 121-123
- orphan, 122
- parent, 111
- passing data between, 813
- periodic, 230
- Pico, 91
- plug-in, 124
- producer, 126-127, 290, 558-559
- renderer, 124
- service, 123
- sibling, 111
- single-threaded, 160
- state of, 107-109
- system, 872
- systemd, 117
- threads performed by, 110
- visible, 122
- in Windows 10, 886
- zombie, 122

process-contention scope (PCS), 217-218

process control blocks (PCBs), 109-110

process-control system calls, 66-71

process identifier (pid), 116, 787

process identity (Linux), 787

Process Lifetime Manager (PLM), 837**process management:**

- about, 27-28
- in Linux, 786-790
- Windows 10, 886-891

process manager (Windows 10), 858-860**process migration, 752-753****process name, 73****processors, 18**

- distributed system, 734
- front-end, 522
- ideal, 242, 842
- Intel, 379-382
 - event-vector table, 496
 - IA-32 architecture, 379-382
 - IA-64 architecture, 382
 - thread building blocks, 186-188
- logical, 832
- multi-, 18, 220
- multicore, 221-224

processor affinity, 225-226**processor groups, 832****process reflection, 860****process representation (Linux), 111****process scheduler, 110-112****process scheduling, 110-115, 199, 234****process synchronization, 260. See also synchronization tools****process termination, deadlock recovery by, 342****/proc file system (Linux), 808-810****procfs (file system), 598****producer process, 126-127, 290, 558-559****production kernels (Linux), 777****program counters, 27, 106, 109****program execution (operating system service), 56****program loading and execution, 74****programmable interval timer, 505****programmed I/O (PIO), 498****programmer interface (Windows 10), 884-895**

- IPC with Windows messaging, 891-892
- kernel object access, 884-885
- memory management, 892-895
- process management, 886-891
- sharing objects between processes, 885-886

programming:

- multi-, 23, 112, 420
- multicore, 162-166

programming-environment

- virtualization, 703, 717

programming languages, 313-314

- programming-language support, 74
- program structure, for demand paging, 433-434
- program threats, 625-634
 - code injection, 628-631
 - malware, 625-628
 - viruses, 631-634
 - worms, 631, 632
- progress (requirement), 260
- projects, 244
- proportional allocation, 414-415
- proportional share scheduling, 233
- proprietary software, 46-47
- protection, 667-698
 - access matrix model, 675-685
 - implementation, 679-682
 - mandatory access control, 684-685
 - and revocation of access rights, 682-683
 - role-based access control, 683-684
 - capability-based systems, 685-687
 - code signing, 690
 - in computer systems, 33-34
 - with contiguous memory allocation, 357
 - domain of, 671-675
 - file, 531
 - file-system interface, 550-555
 - goals of, 667-668
 - I/O, 512
 - language-based systems, 690-696
 - compiler-based enforcement, 691-693
 - in Java, 694-696
 - as operating system service, 57-58
 - in paged environment, 368-369
 - and principle of least privilege, 668-669
 - rings of, 669-671
 - sandboxing, 689-690
 - static vs. dynamic, 673
 - system-call filtering, 688
 - system integrity, 687-688
 - from viruses, 657-659
- protection domains, 671-675, 711
- protection mask (Linux), 817
- protection rings, 25, 669-671
- protection system calls, 73-74
- pseudo-device driver, 721-722
- PTBR (page-table base register), 365
- PTEs (page-table entries), 853
- PTE tables, 853
- Pthreads, 169-171, 218-219
- PTLR (page-table length register), 369
- P-to-V conversion, 724
- public cloud, 44
- public domain software, 779-780

- public keys, 641
- public-key encryption, 641
- pull migration, 224
- pure code (reentrant), 370
- pure demand paging, 395
- pushing, 66, 519
- Pushlocks, 831
- push migration, 224
- put command, 751

Q

- Quest-V, 729
- queue(s), 38
 - dispatch, 185
 - main, 185
 - ready, 112, 221, 843
 - scheduling, 112-113
 - wait, 112
- queuing diagram, 112, 113
- queuing-network analysis, 247

R

- race condition, 259, 261
- RAID (redundant arrays of inexpensive disks), 473-485
 - extensions, 481-482
 - levels of, 475-481
 - for object storage, 483-485
 - performance improvement, 475
 - problems with, 482-483
 - reliability improvement, 473-475
 - structuring, 474
- RAID level 0, 476
- RAID level 0 + 1, 478-479
- RAID level 1, 476
- RAID level 1 + 0, 478-479
- RAID level 4, 476-477
- RAID level 5, 477-478
- RAID level 6, 478
- RAID levels, 475-481
 - common, 475-478
 - selecting, 480-481
 - variations in, 478-480
- raising interrupts, 9, 494
- RAM (random-access memory), 11
- RAM drives, 454, 455
- random-access devices, 502
- random-access memory (RAM), 11
- random-access time (disks), 450
- range (value), 187
- ransomware, 626
- RAT (Remote Access Tool), 625

- rate-monotonic scheduling, 230-232
- rate-monotonic scheduling algorithm, 230-232
- raw disk, 413, 464, 601
- raw I/O, 464, 465, 503-504
- raw partitions, 468
- RBAC (role-based access control), 683-684
- RDP, 707
- reacquisition, of capabilities, 682
- read access, locks with, 292
- read-ahead technique, 585
- read-end (of pipe), 140
- readers, 291, 292
- readers-writers problem, 290-293
- reader-writer locks, 292-293
- reading files, 532, 551
- read-modify-write cycle, 477
- read only devices, 502
- read pointer, 532
- read-write devices, 502
- ready queues, 112, 221, 843
- ready state, 108, 109
- real-time class, 239
- real-time CPU scheduling, 227-234
 - earliest-deadline-first scheduling, 232-233
 - and minimizing latency, 227-229
 - POSIX real-time scheduling, 233-234
 - priority-based scheduling, 229-230
 - proportional share scheduling, 233
 - rate-monotonic scheduling, 230-232
- real-time embedded systems, 45-46
- real-time operating systems, 46
- real-time range (Linux schedulers), 790
- real-time scheduling (Linux), 792
- reapers, 417-418
- receives, blocking vs. nonblocking, 130
- reconfiguration 755
- records:
 - activation, 107
 - base file, 876
 - logical, 539
 - master boot, 465, 466
- recovery:
 - from deadlock, 341-343
 - from failure, 755-756
 - and file system implementation, 586-589
 - Windows 10, 877-878
- recovery mode, 95
- red-black trees, 38, 40
- Red Hat, 779
- redirectors, 882-883
- redundancy, 473-475
- redundant arrays of inexpensive disks, see RAID
- reentrant code (pure code), 370
- reentrant locks, 307-308
- reference, locality of, 395
- reference bits, 409
- referenced pointer, 849
- reference string, 404, 406
- reflection process, 860
- reflecto , 864
- regions, 383
- register(s):
 - base, 351-352
 - control, 492
 - CPU, 110
 - data-in, 492
 - data-out, 492
 - instruction, 12
 - limit, 351-352
 - memory-address, 354
 - page-table base, 365
 - page-table length, 369
 - relocation, 354
 - status, 492
 - translation table base, 383
- registry, 74, 871-872
- regression testing, 249
- relative access, 539-540
- relative block number, 540
- relative path names, 546
- release, of resources, 318
- reliability:
 - of distributed systems, 735
 - RAID for improving, 473-475
 - of TCP, 748
 - of UDP, 747
 - of Windows 10, 828-829
- relocatable code, 353
- relocatable object file, 75
- relocation, 75, 76
- relocation register, 354
- remainder section, 260
- Remote Access Tool (RAT), 625
- remote desktop, 874
- remote file access, 764-767
- remote file-systems 605-608
- remote file transfer, 750-751
- remote login, 750
- remote operations (NFS), 615
- remote procedure calls (RPCs), 149-153, 834
- remote-service mechanism, 764
- removable storage media, 451
- remove() method, 305-307

- renaming files 542
- renderer processes, 124
- rendezvous, 131
- repair, mean time to, 474
- replacement, page, *see* page replacement
- replay attacks, 622
- replication, 480, 592-593
- reply port, 135
- repositioning (in files) 532
- Request (data structure), 335-336, 339, 340
- requests, for resources, 318
- request consumer (circular buffer), 716
- request edge, 323
- request manager, 811
- request producer (circular buffer), 716
- resident attributes, 876
- resolution:
 - address resolution protocol, 745
 - conflict, 784, 785
 - of links, 548
 - name, 738-741
 - and page size, 431-432
- resource allocation (operating system service), 57
- resource-allocation graph, 323-326, 334, 338
- resource-allocation-graph algorithm, 333
- resource allocator, operating system as, 5
- resource arbiters, 869
- resource management, 27-32
- resource preemption, deadlock recovery by, 342-343
- resource-request algorithm, 335
- resource sharing, 162, 734-735
- resource utilization, 4
- responses (password), 652
- response consumer (circular buffer), 716
- response producer (circular buffer), 716
- response time, 23, 205
- responsibility, for run-time-based enforcement, 694
- responsiveness, multithreaded process, 162
- restart area, 878
- restore, state, 114
- restore point, system, 871
- restoring data, 588-589
- resuming, 717, 888
- return from sleep, 243
- reverse engineering, 47
- revocation of access rights, 682-683
- RHEL 7, 461
- rich text format (RTF), 658
- right child, 38

- rights:
 - access, 534, 673, 680, 682-683
 - copy, 677
 - group, 817
 - owner, 678, 817
 - port, 135
 - user, 817
 - world, 817
- rings, protection, 669-671
- risk assessment, 653-654
- roaming profiles 883
- robustness, distributed system, 754-756
- roles, 683
- role-based access control (RBAC), 683-684
- rollback, 343
- root directory, 877
- rootkit viruses, 632
- root partition, 601
- rotational latency (disks), 451
- rotations per minute (RPM), 450
- round robin, 130
- round-robin (RR) scheduling algorithm, 209-211
- routers, 736
- RPCs (remote procedure calls), 149-153, 834
- RPM (rotations per minute), 450
- RR scheduling algorithm, 209-211
- RSA algorithm, 641, 642
- RTE (run-time environment), 64-65
- RTF (rich text format), 658
- running state, 108, 109
- running system, 94
- run time, virtual, 236
- run-time-based enforcement, 694-696
- run-time environment (RTE), 64-65

S

- SaaS (software as a service), 44
- safe computing, 658
- safe sequence, 331
- safe state, 331-333
- safety, as virtualization requirement, 704
- safety algorithm, 335
- sandbox, 124, 658
- sandboxing, 689-690
- SANs, *see* storage-area networks
- SAS buses, 456, 490
- SATA buses, 456
- save, 114, 592
- scalability, 162, 484, 756-757
- SCAN scheduling, 458-459

- SCAN (elevator) scheduling algorithm, 458-459**
- scatter-gather method, 498, 508**
- schedulers:**
 - CFQ, 461, 811
 - Completely Fair, 236, 237, 790
 - CPU, 113-114, 201
 - deadline, 460, 461
 - Linux, 790
 - NOOP, 461
 - process, 110-112
- scheduler activation, 192-194**
- scheduling:**
 - cooperative, 202
 - CPU, *see* **CPU scheduling**
 - C-SCAN, 460
 - earliest-deadline-first, 232-233
 - fair, 791
 - FCFS, 458, 459
 - HDD, 457-461
 - I/O, 508-509
 - job, 106
 - in Linux, 790-794
 - multi-processor, *see* **multi-processor scheduling**
 - nonpreemptive, 202
 - NVM, 461-462
 - preemptive, 202-203
 - priority-based, 229-230
 - process, 110-115, 199, 234
 - proportional share, 233
 - rate-monotonic, 230-232
 - real-time, 792
 - SCAN, 458-459
 - selecting disk-scheduling algorithm, 460-461
 - shortest-remaining-time-first, 209
 - thread, 199, 790-791, 844-845
 - user-mode, 241, 833, 890-891
 - in Windows 10, 887
- scheduling classes, 236**
- scheduling context, 788**
- scheduling domain, 238**
- scheduling information, CPU, 110**
- scheduling rules, 887**
- SCM (service control manager), 870**
- scope:**
 - contention, 217-218
 - of lock, 305
- script kiddies, 631**
- scripts, shell, 61, 536**
- SCS (system-contention scope), 218**
- searching, for files, 541, 542**
- search path, 545**
- secondary memory, 395**
- secondary storage, 13. *See also* **disk(s)****
 - allocation of, 570-578
 - contiguous allocation, 570-573
 - indexed allocation, 575-577
 - linked allocation, 573-575
 - and performance, 578-579
 - connection methods for, 456
- second-chance page-replacement algorithm (clock algorithm), 410-411**
- second extended file system (ext2), 805**
- second-level interrupt handler (SLIH), 496**
- second readers, 291**
- section objects, 139, 852**
- sectors, 450, 466, 566**
- sector slipping, 467**
- sector sparing, 466**
- Secure Boot, 872**
- secure by default, 634**
- secure kernel, 839-840**
- Secure Monitor Call (SMC), 670**
- secure shell, 116**
- secure system process, 872**
- secure systems, 622**
- Secure World, 838**
- security, 621-665. *See also* **protection****
 - of compiler-based enforcement, 692
 - in computer systems, 33-34
 - cryptography for, 637-648
 - and encryption, 638-645
 - implementation, 645-646
 - TLS example, 646-648
 - implementation of, 653-662
 - and accounting, 659
 - and auditing, 659
 - and firewalling, 659-660
 - and intrusion prevention, 655-657
 - levels of defenses, 661-662
 - and logging, 659
 - and security policy, 653
 - and virus protection, 657-659
 - and vulnerability assessment, 653-655
 - in Linux, 816-818
 - as operating system service, 57-58
 - as problem, 621-625
 - and program threats, 625-634
 - code injection, 628-631
 - malware, 625-628
 - viruses, 631-634
 - worms, 631, 632
 - and system/network threats, 634-637
 - user authentication for, 648-653

- in Windows 10, 662-664, 826-828, 878
- security access tokens (Windows 10), 662**
- security context (Windows 10), 662-663**
- security descriptor (Windows 10), 663**
- security domains, 659**
- security ID (SID), 33, 867**
- security policy, 653**
- security reference monitor (SRM), 866-869**
- security-through-obscure approach, 655**
- security tokens, 867**
- seek, file, 532**
- seek time (disks), 450**
- segmentation, IA-32, 379-380**
- selective revocation, 682**
- semantics, 510, 608-609**
- semaphore(s), 272-276**
 - binary, 273
 - counting, 273
 - defined, 272
 - dining-philosophers solution with, 294-295
 - implementation, 274-276
 - in Java, 308-309
 - monitors using, 280-281
 - named, 300-301
 - POSIX examples, 300-302
 - unnamed, 300-302
 - usage of, 273-274
- semaphore objects (Windows 10), 845**
- semiconductor memory, 14**
- sends, blocking vs. nonblocking, 130**
- sending messages, 891**
- sense key, 512**
- separation hypervisors, 729**
- sequence numbers, 748**
- sequential access (files) 539, 541**
- sequential devices, 502**
- serial ATA (SATA) buses, 456**
- serial-attached SCSI (SAS) buses, 456, 490**
- serial dispatch queue, 185**
- server(s), 73**
 - blade, 18-19
 - bootstrap, 136
 - in client-server model, 606, 758-759, 861-862
 - defined, 757
 - in distributed systems, 734
 - file-server systems, 43
 - name, 739
 - and redirectors, 882-883
- server-initiated approach to verifying cached data, 767**
- Server Message Block (SMB), 880**
- server subject (Windows 10), 663**
- server systems, 42-43, 734, 874-875**
- service(s):**
 - defined, 757
 - denial of, 622, 636
 - distributed naming, 607
 - high-availability, 19
 - infrastructure as, 44
 - log-file, 878
 - network information, 607
 - operating system, 55-58, 74-75, 115, 152
 - platform as, 44
 - software as, 44
 - theft of, 622
- service control manager (SCM), 870**
- service processes, 123**
- service-trigger mechanism, 870**
- session(s), 751, 874**
- session 0, 873**
- session hijacking, 623**
- session key, 647**
- session layer, 742**
- session manager subsystem (SMSS), 872-873**
- session semantics, 609**
- sets:**
 - entry, 303, 304, 307
 - hard working-set limits, 438
 - of holes, 358
 - SMT, 242
 - wait, 304, 307
 - working, 422-424, 438
- setuid attribute, 34**
- setuid bit, 674-675**
- SHA-1 message digest, 643**
- shadow copies, 863**
- sharable devices, 502**
- shares, 244**
- shared directories, 547**
- shared files 609**
- shared libraries, 356, 392**
- shared lock, 534**
- shared memory, 123, 125, 556-560**
- shared-memory model, 57, 73, 125-127, 132-136**
- shared ready queue, 843**
- shared system interconnect, 18**
- sharing:**
 - file, 602-603
 - information, 123
 - load, 220
 - and paging, 369-371
 - resource, 162, 734-735
 - space, 592
- sharing objects, 885-886**

- shells, 58, 116, 783
- shell scripts, 61, 536
- short duration locks, 272
- shortest-job-firs (SJF) scheduling
 - algorithm, 207-209
- shortest-next-CPU-burst algorithm, 207
- shortest-remaining-time-firs scheduling, 209
- shoulder surfing, 649
- sibling process, 111
- SID (security ID), 33, 867
- Siemens Jailhouse project, 729
- signals, 188-189, 812-813
- signal-and-continue method, 279
- signal-and-wait method, 279
- signaled state, 297
- signal handlers, 188-189
- signal-handler table, 789
- signatures, 633, 643, 656, 828
- signature-based detection, 656
- silos, 859
- SIMD, 833
- simple messages, 136
- simple subject (Windows 10), 662
- simulations, 248-249
- simultaneous multithreading, 222
- single indirect blocks, 576
- single-level directories, 542-543
- single-processor systems, 15-16
- single step (mode), 72
- single-threaded processes, 160
- single-user mode, 95
- singly linked lists, 37
- SIP (System Integrity Protection), 687-688
- Siri, 5
- sites, distributed system, 734
- 64-bit computing, 383
- SJF scheduling algorithm, 207-209
- sketch, 70
- slabs, 427-429, 797-798
- slab allocation, 427-430, 797-798
- Slackware, 779
- sleep, return from, 243
- SLIH (second-level interrupt handler), 496
- slim reader-writer (SRW) locks, 889
- SLOB allocator, 430
- SLUB allocator, 430
- small objects, 430
- SMB (Server Message Block), 880
- SMC (Secure Monitor Call), 670
- SMP, see symmetric multiprocessing
- SMSS (session manager subsystem), 872-873
- SMT sets, 242
- snapshots, 480, 588, 705, 879
- sniffing, 635-636, 649-650
- social engineering, 624
- sockets, 146-149
- socket interface, 504
- soft affinity, 225
- soft errors, 463
- soft page faults, 416
- soft real-time systems, 227
- software:
 - process migration and, 753
 - proprietary, 46-47
 - public domain, 779-780
- software as a service (SaaS), 44
- software engineering, 80
- software interrupts (traps), 497
- software objects, 672
- software transactional memory (STM), 312
- Solaris, 51
 - file systems in, 482-484, 597, 599
 - Oracle SPARC, 375-376
 - scheduling example, 242-244
 - virtual memory in, 438-440
 - ZFS file system, 482-484, 581, 588, 598
- Solaris 10:
 - role-based access control in, 683, 684
 - zones in, 718, 719
- solid-state disks (SSDs), 452
- source-code viruses, 633
- source files, 530
- space maps, 581
- space sharing, 592
- SPARC, 375-376
- sparseness, 374, 391
- special instructions, 709
- special-purpose file systems, 597-598
- specifications thread behavior, 169
- speed of operations (I/O devices), 502
- spinlocks, 272
- split-screen, 115
- spoofed identifiers, 606
- spoofing, 636
- spools, 511
- spooling, 511
- Springboard interface, 60, 87
- spyware, 626
- SRM (security reference monitor), 866-869
- SRW (slim reader-writer) locks, 889
- SSDs (solid-state disks), 452
- stacks, 37-38, 66
 - device, 862
 - LRU page replacement with, 408

- OSI protocol, 742-744
- stack algorithms, 408-409
- stack inspection, 694, 695
- stack section (of process), 107
- stalling, 350
- standard swapping, 377
- standby page, 856
- starvation (indefinite blocking), 213, 343
- states:
 - application, 378
 - new, 108
 - nonsignaled vs. signaled, 297
 - of processes, 107-109
 - ready, 108, 109
 - running, 108, 109
 - safe, 331-333
 - suspended, 705, 888
 - terminated, 109
 - unsafe, 332-334
 - waiting, 108, 109
- state information, 608
- stateless DFS, 608
- state restore, 114
- state save, 114
- static linking, 355-356, 803
- static protection, 673
- status information, 74
- status register, 492
- stealth viruses, 633
- STM (software transactional memory), 312
- storage, 11-14. See also mass-storage
 - structure
 - cloud, 471, 751
 - content-addressable, 484
 - definitions and notations, 12
 - host-attached, 470
 - network-attached, 470-471
 - nonvolatile, 14, 449
 - object, 483-485
 - secondary, 13, 456, 570-578. See also **disk[s]**
 - tertiary, 13
 - thread-local, 192, 894, 895
 - utility, 481
 - volatile, 11
- storage-area networks (SANs), 21, 470, 472
- storage array, 472-473, 481
- storage attachment, 469-473
- storage devices, organization of, 597, 598
- storage device management, 463-467
- storage management, 30, 32, 723
- stream ciphers, 640
- stream head, 519
- streaming transfer rate, 486
- stream modules, 519
- STREAMS mechanism, 519-521
- string, reference, 404, 406
- strongly ordered model, 265
- strong passwords, 651
- stubs, 150
- subjects (Windows 10), 662-663
- subsets, stack algorithm, 408
- subsystems, 75
- SunOS, 51
- superblock, 566
- superblock objects, 605, 804, 805
- supervisor mode, see **kernel mode**
- SuSE, 779
- suspended state, 705, 888
- swap map, 469
- swapping, 113-114, 376-378
 - in Linux, 800
 - on mobile systems, 377-378
 - with paging, 377, 378
 - standard, 377
 - system performance under, 378
- swap space, 395, 468-469
- swap-space management, 467-469
- SwitchBranch mechanism, 830
- switches, context, 114-115, 204
- switching:
 - domain, 673, 674
 - fast-user, 825, 874-875
- symbolic links, 879
- symmetric clustering, 20
- symmetric encryption, 639-640
- symmetric encryption algorithm, 639
- symmetric multiprocessing (SMP), 16, 220, 794
- symmetry, in addressing, 129
- synchronization, 130-131, 289-314
 - alternative approaches to, 311-314
 - block, 305
 - bounded-buffer problem, 290
 - dining-philosophers problem, 293-295
 - for interprocess communication, 812-813
 - in Java, 303-311
 - condition variables, 309-311
 - monitors, 303-307
 - reentrant locks, 307-308
 - semaphores, 308-309
 - kernel, 295-299, 792-794
 - in Linux, 130-131, 812-813
 - in message-passing model, 130-131
 - in POSIX, 299-303
 - process, 260. See also **synchronization tools**

- readers-writers problem, 290-293
 - thread, 888-889
 - synchronization primitives, 845-846**
 - synchronization tools, 257-287**
 - about, 257-260
 - critical-section problem, 260-270
 - hardware solution to, 265-270
 - Peterson's solution to, 262-265
 - evaluation of, 284-286
 - and liveness, 283-284
 - monitors for, 276-282
 - resumption of processes within, 281-282
 - semaphores, implementation using, 280-281
 - usage, 277-280
 - mutex locks, 270-272
 - semaphores for, 272-276
 - synchronous devices, 502, 506, 507**
 - synchronous message passing, 130**
 - synchronous threading, 169**
 - synchronous writes, 585**
 - system administrators, 60**
 - system build, 92**
 - system calls (monitor calls), 22, 62-74**
 - and API, 63-66
 - clone(), 195-196
 - for communication, 72-73
 - for device management, 71-72
 - exec(), 188
 - for file management, 71
 - fork(), 188
 - functioning of, 62-63
 - for information maintenance, 72
 - for I/O, 512, 513
 - for process control, 66-71
 - for protection, 73-74
 - system-call filtering, 688**
 - system-call firewalls, 660**
 - system-call interface, 65**
 - system components (Windows 10), 838-874**
 - executive, 848-874
 - hardware-abstraction layer, 840
 - hyper-V hypervisor, 839
 - kernel, 840-848
 - secure kernel, 839-840
 - system-contention scope (SCS), 218**
 - system daemons, 22, 781**
 - system-development time, 705**
 - system disk, 465**
 - systemd process, 117**
 - system goals, 79**
 - System Integrity Protection (SIP), 687-688**
 - system libraries (Linux), 781**
 - system mode, see kernel mode**
 - system model, for deadlocks, 318-319**
 - system processes, 872**
 - system programs, 6**
 - system resource-allocation graph, 323-326**
 - system restore point, 871**
 - system utilities, 74-75, 781**
 - System V init, 117**
 - system-wide open-file table, 567**
 - system-wide tools, 96, 97**
- T**
- table(s). See also page table(s)**
 - attribute-definition, 877
 - device-status, 508-509
 - event-vector, 11
 - file, 788
 - file-allocation, 574-575
 - frame, 365
 - global, 679
 - global descriptor, 379
 - handle, 849
 - hash, 570
 - master file, 566
 - mount, 517, 567
 - open-file, 533, 567
 - page directory pointer, 381
 - per-process open-file, 567
 - PTE, 853
 - signal-handler, 789
 - system-wide open-file, 567
 - tags, 680**
 - tapes, magnetic, 455**
 - target latency, 236, 791**
 - target thread, 190**
 - tasks, 106, 135, 195, 234. See also user programs (user tasks)**
 - task control blocks, see process control blocks**
 - task identification for multicore programming, 163**
 - task parallelism, 165, 166**
 - Task Self port, 135**
 - TB (terabyte), 12**
 - TBBs (thread building blocks), 186-188**
 - TCP (transmission control protocol), 743-749**
 - TCP/IP, see Transmission Control Protocol/Internet Protocol**
 - TCP sockets, 147**
 - TDI (Transport Driver Interface), 880**

- TEBs (thread environment blocks), 889-890
- templating, 706
- temporary revocation, 682
- terabyte (TB), 12
- terminal concentrators, 522
- terminal server systems, 874-875
- terminated state, 109
- termination, 121-123, 342
- tertiary storage devices, 13
- testing, multicore programming in, 165
- text files 530
- text section (of process), 106
- theft of service, 622
- thin-client computing, 874-875
- thin clients, 40
- third extended file system (ext3), 805-807
- 32-byte memory, 363, 364
- thrashing, 419-425
 - cause of, 419-422
 - current practice, 425
 - and page-fault-frequency strategy, 424-425
 - and working-set model, 422-424
- threads, 159-197.** See also **threading**
 - alertable, 846
 - green, 167
 - hardware, 222
 - idle, 239, 842
 - Java, 173-176
 - kernel, 166, 217, 234
 - kernel-mode, 841
 - in Linux, 789-790
 - and multicore programming, 162-166
 - in operating systems, 194-196
 - and process model, 110
 - Pthreads, 169-171, 218-219
 - scheduling of, 199
 - target, 190
 - user, 166, 217
 - user-mode, 841
 - in Windows 10, 841-845, 886-889, 894
- Thread attach, 860**
- thread building blocks (TBBs), 186-188**
- thread cancellation, 190-192**
- thread dumps, 339**
- thread environment blocks (TEBs), 889-890**
- threading:**
 - asynchronous, 169
 - hyper-, 222, 832
 - implicit, 176-188
 - fork join, 180-183
 - Grand Central Dispatch, 185-186
 - Intel thread building blocks, 186-188
 - OpenMP and, 183-185
 - thread pools and, 177-180
 - issues:
 - fork() and excel() system calls, 188
 - scheduler activations, 192-194
 - signal handling, 188-190
 - thread cancellation, 190-192
 - thread-local storage, 192
 - multi-, 166-168, 222, 223
 - synchronous, 169
- thread libraries, 168-176**
 - about, 168-169
 - Java, 173-176
 - Pthreads, 169-171
 - Windows, 171-173
- thread-local storage (TLS), 192, 894, 895**
- thread pools, 177-180, 889**
- thread scheduling, 199**
 - in Linux, 790-791
 - in Windows 10, 844-845
- threats, 622**
 - program, 625-634
 - code injection, 628-631
 - malware, 625-628
 - viruses, 631-634
 - worms, 631, 632
 - system/network, 634-637
- three-way handshake, 748**
- throughput, 204-205**
- thunking, 830**
- tightly coupled systems, 83**
- time:**
 - compile, 352
 - down, 572
 - effective access, 397-398
 - effective memory-access, 367
 - execution, 353
 - load, 353
 - mean time between failures, 473, 474
 - mean time of data loss, 474
 - mean time to repair, 474
 - positioning, 450
 - random-access, 450
 - response, 23, 205
 - seek, 450
 - system-development, 705
 - turnaround, 205
 - virtual run, 236
 - waiting, 205
- time quantum, 209-211, 243**
- time quantum expired, 243**
- timers, 26-27, 505-506**
- timer objects, 845**

- time slice, 209-211, 790-791
 - timestamps, 531
 - timestamp counters (TSCs), 845
 - TLB, *see* translation look-aside buffer
 - TLB miss, 366
 - TLB reach, 432-433
 - TLB walk, 376
 - TLS (thread-local storage), 192, 894, 895
 - TLS (Transport Layer Security), 646-648
 - tmpfs (fil system), 598
 - top half (interrupt service routines), 793-794
 - total revocation, 682
 - touch screens, 5
 - touch-screen interface, 56, 60
 - trace files, 248
 - tracing tools, 97-98
 - tracks, disk, 450
 - traditional computing, 40-41
 - traffic network, 635-636
 - transactions, 311, 587, 808
 - transactional memory, 311-312
 - transfer rates, 450, 451, 486
 - transition page, 856
 - translation:
 - binary, 708-710
 - flash translation layer, 453-454
 - network address, 723
 - path-name, 614-615
 - translation granules, 383
 - translation look-aside buffer (TLB), 365-368, 376, 384, 855
 - translation table base register, 383
 - transmission control protocol (TCP), 743-749
 - Transmission Control Protocol/Internet Protocol (TCP/IP), 36, 743-746, 880-881
 - transparency, 756, 761
 - Transport Driver Interface (TDI), 880
 - transport layer, 742
 - transport-layer protocol (TCP), 645
 - Transport Layer Security (TLS), 646-648
 - traps, 22, 89, 497, 847
 - trap-and-emulate method, 707-708
 - trap doors, 626, 627
 - traversing fil system, 542
 - trees, 38, 39, 116
 - tree-structured directories, 545-547
 - TRIMing unused blocks, 581-582
 - trimming, automatic working-set, 438
 - triple DES, 639
 - triple indirect blocks, 576, 577
 - Trojan horses, 625-626
 - truncating files, 532
 - trusted addresses, 638
 - Trustlets, 838
 - TrustZone (TZ), 670, 671
 - TSCs (timestamp counters), 845
 - tunneling, attacks with, 659-660
 - turnaround time, 205
 - twisted pair cables, 736
 - two-factor authentication, 652
 - two-level directories, 543-545
 - two-level model, 168
 - two-level page-table scheme, 372-373
 - type 0 hypervisors, 702, 713-714
 - type 1 hypervisors, 703, 714-715
 - type 2 hypervisors, 703, 715-716
 - type safety (Java), 696
 - TZ (TrustZone), 670, 671
- U**
- UDP (user datagram protocol), 743, 746-748
 - UDP sockets, 147
 - UEFI (Unifire Extensible Firmware Interface), 94
 - UFD (user fil directory), 543
 - UFS (UNIX fil system), 565-566, 598
 - UI, *see* user interface
 - UMDF (User-Mode Driver Framework), 864
 - UMS, *see* user-mode scheduling
 - unbounded buffer, 126
 - unbounded capacity (of queue), 132
 - UNC (Uniform Naming Convention), 881
 - uncontended loads, 285
 - uncontended locks, 271
 - unifire buffer cache, 583-585
 - Unifire Extensible Firmware Interface (UEFI), 94
 - unifire virtual memory, 583
 - Uniform Naming Convention (UNC), 881
 - unikernels, 728
 - universal serial buses (USBs), 456
 - Universal Windows Platform (UWP), 426
 - UNIX fil system (UFS), 565-566, 598
 - UNIX operating system:
 - consistency semantics, 609
 - inode, 577
 - I/O kernel structure in, 513, 514
 - permissions in, 553
 - protection domain in, 674-675
 - system calls, 68
 - system structure, 82
 - unloader, module, 783

- unnamed data, 875
- unnamed semaphores, 300-302
- unsafe state, 332-334
- unstructured data, 484
- untrusted applet protection, 695
- upcalls, 193
- upcall handler, 193
- updating policy, cache, 766-767
- urgency value, 223
- URL loader, 695
- USB drive, 452
- USBs (universal serial buses), 456
- use, of resources, 318
- users, 4-5, 603
 - as domains, 674
 - multiple, file sharing between, 602-603
 - other users (class), 551
 - power, 60-61
- user accounts, 662
- user authentication, 648-653
- user control list, 561
- user datagram protocol (UDP), 743, 746-748
- user-define signal handlers, 189
- user experience layer (macOS and iOS), 87
- user file directory (UFD), 543
- user goals, 79
- user IDs, 33, 531, 675
- user-initiated class, 185-186
- user-interactive class, 185
- user interface (UI), 4-5, 56, 58-62
- user mode, 24, 25, 782
- User-Mode Driver Framework (UMDF), 864
- user-mode scheduling (UMS), 241, 833, 890-891
- user-mode threads (UT), 841
- user programs (user tasks), 106, 353, 801-803
- user rights (Linux), 817
- user threads, 166, 217
- UT (user-mode threads), 841
- utility class, 186
- utility storage, 481
- UWP (Universal Windows Platform), 426

V

- VACB (virtual address control block), 865
- VADs (virtual address descriptors), 857
- valid-invalid bit, 368-369
- valid page, 856
- variables:

- atomic, 269-270
 - condition, 278, 279, 302-303, 309-311, 889
- variable class, 239
- variable-partition schemes, 358
- variable timer, 26
- VCPU (virtual CPU), 707
- vectored I/O, 507-508
- verifie , 98
- version control system, 49
- vfork() (virtual memory fork), 400
- VFS (virtual file system), 804-805
- VFS layer, 601
- victim, for resource preemption, 343
- victim frames, 402
- views, 557, 852
- virtual address, 354
- virtual address control block (VACB), 865
- virtual address descriptors (VADs), 857
- virtual address space, 390, 391, 799-800
- VirtualBox project, 704
- virtual CPU (VCPU), 707
- virtual file-systems, 603-605
- virtual file system (VFS), 804-805
- virtual file system (VFS) layer, 601
- virtualization, 34-35
 - defined, 701
 - operating-system components for, 719-726
 - CPU scheduling, 720
 - I/O devices, 722-723
 - live migration, 724-726
 - memory management, 721-722
 - storage management, 723
 - para-, 703, 716-717
 - programming-environment, 703, 717
 - research, 728-729
- virtual machines, 34, 701-730. See also **virtualization**
 - benefits of, 704-707
 - building blocks, 707-713
 - binary translation, 708-710
 - hardware assistance, 710-713
 - trap-and-emulate method, 707-708
 - examples, 726-728
 - features of, 704-707
 - history of, 703-704
 - implementations, 713-719
 - application containment, 718-719
 - emulation, 717-718
 - paravirtualization, 716-717
 - programming-environment
 - virtualization, 717
 - type 0 hypervisors, 713-714
 - type 1 hypervisors, 714-715

- type 2 hypervisors, 715-716
 - and virtual machine life cycle, 713
 - life cycle, 713
- virtual machine control structures (VMCSs), 711**
- virtual machine managers (VMMs), 25-26, 35, 702**
- virtual machine sprawl, 713**
- virtual memory, 24, 389-441**
 - background on, 389-392
 - and copy-on-write technique, 399-401
 - demand paging for conserving, 392-399, 430-436
 - basic mechanism, 393-396
 - free-frame list, 396-397
 - with inverted page tables, 433
 - and I/O interlock, 434-436
 - and page size, 431-432
 - and performance, 397-399
 - and prepaging, 430-431
 - and program structure, 433-434
 - and TLB reach, 432-433
 - direct virtual memory access, 500
 - and frame allocation, 413-419
 - allocation algorithms, 414-415
 - global vs. local allocation, 415-418
 - minimum number of frames, 413-414
 - non-uniform memory access, 418-419
 - kernel, 801
 - and kernel memory allocation, 426-430
 - in Linux, 798-801
 - and memory compression, 425-426
 - network, 765
 - operating-system examples, 436-440
 - page replacement for conserving, 401-413
 - and application performance, 412-413
 - basic mechanism, 401-404
 - counting-based page replacement, 411-412
 - FIFO page replacement, 404-406
 - LRU-approximation page replacement, 409-411
 - LRU page replacement, 407-409
 - optimal page replacement, 406-407
 - and page-buffering algorithms, 412
 - and thrashing, 419-425
 - cause, 419-422
 - current practice, 425
 - page-fault-frequency strategy, 424-425
 - working-set model, 422-424
 - unified, 583
 - in Win32 API, 892, 893
- virtual memory context, 789**
- virtual memory fork, 400**
- virtual memory (VM) manager, 851-858**
- virtual memory regions, 799**
- virtual private networks (VPNs), 646, 881**
- virtual run time, 236**
- virtual to physical (V-to-P) conversion, 724**
- Virtual Trust Levels (VTLs), 838**
- virus dropper, 632**
- viruses, 631-634, 657-659**
- virus signatures, 633**
- visible processes, 122**
- VMCSs (virtual machine control structures), 711**
- VM manager, 851-858**
- VMMs, see virtual machine managers**
- VMware, 704, 726-727**
- vnode, 604**
- voice over IP (VoIP), 44**
- voice recognition, 5**
- volatile memory, 454-455**
- volatile storage, 11**
- volume, 464-465, 474**
- volume control block, 566**
- volume file, 876-877**
- volume shadow copies, 879**
- voluntary context switches, 204**
- von Neumann architecture, 12**
- VPNs (virtual private networks), 646, 881**
- VSM Enclaves, 840**
- VTLs (Virtual Trust Levels), 838**
- V-to-P conversion, 724**
- VT-x instructions, 710**
- vulnerability assessment, 653-655**

W

- WAFL file system, 589-593**
- wait-for graph, 337, 338**
- waiting, busy, 272**
- waiting state, 108, 109**
- waiting time, 205**
- wait() method, 305-307**
- wait queue, 112**
- wait set, 304, 307**
- wait() system call, 119, 121-122**
- WANs, see wide-area networks**
- weakly ordered model, 265**
- wear leveling, 454**
- Web-distributed authoring and versioning (WebDAV), 881**
- well-known ports, 146**
- well-known port numbers, 747**

wide-area networks (WANs), 36, 735, 737-738

WiFi (wireless) networks, 41, 736-737

Win32 API, 884-895

- creating process, 119-120
- IPC with Windows messaging, 891-892
- kernel object access, 884-885
- memory management, 892-895
- process management, 886-891
- shared memory, 556-560
- sharing objects between processes, 885-886

Windows operating system (generally):

- anonymous pipes, 141, 145
- booting from storage device, 466
- interprocess communication example, 138-139
- scheduling example, 239-242
- synchronization within kernels, 296-298
- system calls, 68
- threads, 194-195

Windows 7, 465, 822

Windows 8, 823

Windows 10, 821-896

- access-control list management in, 555
- design principles, 826-838
 - application compatibility, 830-831
 - dynamic device support, 837-838
 - energy efficiency, 836-837
 - extensibility, 833-834
 - international support, 835-836
 - performance, 831-833
 - portability of, 834-835
 - reliability, 828-829
 - security, 826-828
- developments, 823-825
- fast-user switching with, 874-875
- file system, 875-879
- history of, 821-825
- networking, 880-884
- programmer interface, 884-895
 - IPC with Windows messaging, 891-892
 - kernel object access, 884-885
 - memory management, 892-895
 - process management, 886-891
 - sharing objects between processes, 885-886
- security in, 662-664
- system components, 838-874
 - executive, 848-874
 - hardware-abstraction layer, 840
 - hyper-V hypervisor, 839
 - kernel, 840-848

- secure kernel, 839-840

- terminal services, 874-875

- virtual memory in, 437-438

Windows Desktop Bridge, 823

Windows Driver Foundation, 864

Windows executive, 848-874

- booting, 872-874
- cache manager, 864-866
- client-server computing, 861-862
- I/O manager, 862-864
- object manager, 849-851
- plug-and-play manager, 869-870
- power manager, 870-871
- process manager, 858-860
- registry, 871-872
- security reference monitor, 866-869
- virtual memory manager, 851-858

Windows group policy, 884

Windows messaging, 891-892

Windows Store, 823

Windows subsystem for Linux (WSL), 91

Windows Task Manager, 97

Windows thread library, 171-173

Windows Vista, 822

Windows XP, 822

WinRT, 823

Winsock, 891

wired down entries, 366

wireless access points, 736

wireless (WiFi) networks, 41, 736-737

word, 11

Work (data structure), 335, 339, 340

working sets, 422-424, 438

working-set maximum, 438

working-set minimum, 438

working-set model, 422-424

working-set window, 422

Workstation (VMWare), 726-727

work stealing algorithm, 182

world rights (Linux), 817

World Wide Web, 605, 737

worms, 631, 632

worst-fi strategy, 358, 359

writes, synchronous vs. asynchronous, 585

write access, locks with, 292

write amplification, 462

write-anywhere file layout (WAFL) file system, 589-593

write-back caching, 766

write-end (of pipe), 140

write once devices, 502

write-on-close policy, 766-767

write pointer, 532

writers, 291
write-through policy, 766
writing files, 532, 551
WSL (Windows subsystem for Linux), 91

X

x86-64 architecture, 382
XDR (external data representation), 150
Xen, 704, 716-717
XML firewalls, 660
Xtratum, 729

Y

yellow pages, 607

Z

zero capacity (of queue), 131
zero-day attacks, 656
zeroed page, 856
zero-fill-on-demand technique, 397
ZFS file system, 482-484, 581, 588, 598
zombie process, 122
zombie systems, 634, 635
zones, 718, 719, 795

Glossary

50-percent rule A statistical finding that fragmentation may result in the loss of 50 percent of space.

absolute code Code with bindings to absolute memory addresses.

absolute path name A path name starting at the top of the file system hierarchy.

abstract data type (ADT) A programming construct that encapsulates data with a set of functions to operate on that data that are independent of any specific implementation of the ADT.

access matrix An abstract model of protection in which each row represents a domain, each column an object, and each entry a set of access rights.

access right The ability to execute an operation on an object.

access-control list A list of user names allowed to access a file.

acknowledgment packet In networking, a packet sent in response to the successful receipt of a message or packet.

activation record A record created when a function or subroutine is called; added to the stack by the call and removed when the call returns. Contains function parameters, local variables, and the return address.

active directory (AD) The Windows distributed information system, which is the Windows implementation of LDAP.

acyclic graph In directory structure implementation, a structure that contains no cycles (loops).

adaptive mutex A Solaris scheduling feature that starts as a standard spinlock and, if the object is locked and the lock is not held by a thread running on a CPU, blocks and sleeps until the lock is released.

address space layout randomization (ASLR) An operating system technique to avoid code-injection attacks that place memory objects like the stack and heap at unpredictable locations.

address windowing extension (AWE) A Windows mechanism for memory allocation that allows developers to directly request free pages of

RAM from the memory manager and later commit virtual memory on top of those pages.

address-space identifier A part of a TLB entry that identifies the process associated with that entry and, if the requesting process doesn't match the ID, causes a TLB miss for address-space protection.

address-space layout randomization (ASRL) A Windows 7 feature that randomizes process memory addresses to avoid viruses that jump to specific code locations to gain privileges.

admission control In real-time scheduling, a practice whereby the scheduler may not allow a process to start if it cannot guarantee that the task will be serviced by its deadline.

advanced configuration and power interface (ACPI) Firmware common to PCs and servers that manages certain aspects of hardware, including power and device information.

advanced encryption standard (AES) The NIST cipher designed to replace DES and triple DES.

advanced local procedure call (ALPC) In Windows OS, a method used for communication between two processes on the same machine.

advanced technology attachment (ATA) An older-generation I/O bus.

advisory file-lock mechanism A file-locking system in which the operating system does not enforce locking and file access, leaving it to processes to implement the details.

AFS (OpenAFS) A network file system designed at Carnegie Mellon University with the goal of enabling servers to support as many clients as possible.

aging A solution to scheduling starvation that involves gradually increasing the priority of threads as they wait for CPU time.

ahead-of-time (AOT) compilation A feature of the Android RunTime (ART) virtual machine environment in which Java applications are compiled to native machine code when they are installed on a system (rather than just in time, when they are executed).

G-2 Glossary

allocation problem The determination by the operating system of where to store the blocks of a file.

Amazon Elastic Compute Cloud (ec2) An instance of cloud computing implemented by Amazon.

AMD 64 A 64-bit CPU designed by Advanced Micro Devices; part of a class of CPUs collectively known as x86-64.

AMD-V AMD CPU virtualization technology.

analytic evaluation A means of comparing scheduling-algorithm effectiveness by analyzing an algorithm against a workload and assigning it a score.

anomaly detection In intrusion detection, the use of various techniques to detect anomalous behavior that could be a sign of an attack.

anonymous access Remote access that allows a user to transfer files without having an account on the remote system.

anonymous memory Memory not associated with a file. Pages not associated with a file, if dirty and paged out, must not lose their contents and are stored in swap space as anonymous memory.

Apple file system (APFS) The 2017 file system from Apple that is the default on all modern Apple devices; features a rich feature set, space sharing, clones, snapshots, and copy-on-write design.

Apple iOS The mobile operating system created by Apple Inc.

application component In Android, a basic building block that provides utility to an Android app.

application containment A virtualization-like operating system feature that segregates applications from the operating system (examples include Solaris Zones, BSD Jails, and IBM WPARs).

application frameworks layer In the layered macOS and iOS operating system design, the layer that includes Cocoa and Cocoa Touch frameworks, providing an API for Objective-C and Swift programming languages.

application programming interface (API) A set of commands, functions, and other tools that can be used by a programmer in developing a program.

application program A program designed for end-user execution, such as a word processor, spreadsheet, compiler, or Web browser.

application proxy firewall A firewall that understands protocols spoken by applications across a network, accepts connections to a target, and creates connections to that target, limiting and fixing what is sent from the originator.

application state A software construct used for data storage.

arbitrary code guard (ACG) A Windows 7 exploit-mitigation feature.

argument vector In Linux and UNIX, a list containing the command-line arguments used to invoke a process (and available to the process).

ASIC An application-specific integrated circuit (hardware chip) that performs its tasks without an operating system.

assignment edge In a system resource-allocation graph, an edge (arrow) indicating a resource assignment.

asymmetric clustering A configuration in which one machine in a cluster is in hot-standby mode while the other is running applications.

asymmetric encryption algorithm A cipher algorithm in which different keys are used for encryption and decryption.

asymmetric multiprocessing A simple multiprocessor scheduling algorithm in which only one processor accesses the system data structures and others run user threads, reducing the need for data sharing. A boss processor controls the system; the other processors either look to the boss for instruction or have predefined tasks.

asynchronous In I/O, a request that executes while the caller continues execution.

asynchronous procedure call (APC) A facility that enables a user thread to specify a function that is to be called when the user thread receives notification of a particular event.

asynchronous threading Threading in which a parent creates a child thread and then resumes execution, so that the parent and child execute concurrently and independently of one another.

asynchronous write A write that is buffered and written in arbitrary order, with the requesting thread continuing execution after the write is requested.

atomic safe-save In APFS, a feature primitive that performs file, bundle of files, and directory renaming in single atomic operations.

atomic variable A programming language construct that provides atomic operations on basic data types such as integers and booleans.

atomically A computer activity (such as a CPU instruction) that operates as one uninterruptable unit.

attack An attempt to break a computer system's security.

attack surface The sum of the methods available to attack a system (e.g., all of the network ports that are open, plus physical access).

attacker Someone attempting to breach a computer system's security.

attribute In the Windows NTFS file system, one of the elements making up a file. Each file is seen as a structured object consisting of typed attributes, with each attribute an independent byte stream that can be created, deleted, read, and written.

audit trail The collection of activities in a log for monitoring or review.

authentication The process of correctly identifying a person or device. In cryptography, constraining the set of potential senders of a message.

automatic working-set trimming In Windows, a process whereby, if a threshold of minimum free memory is reached, the number of working-set frames is decreased for every process.

automount A network/distributed file system feature in which file systems from remote servers are automatically located and mounted as needed.

autoprobe In Linux, a device driver probe that auto-detects device configuration.

B+ tree A tree data structure in which every path from the root of the tree to the leaf is the same length.

back door A daemon left behind after a successful attack to allow continued access by the attacker. In cryptography, a method of gaining access to encrypted information without first having the secret keys. More generally, a method of passing arbitrary commands or information when an interface does not provide a standard method.

background Describes a process or thread that is not currently interactive (has no interactive input directed to it), such as one not currently being used by a user. In the Grand Central Dispatch Apple OS scheduler, the scheduling class representing tasks that are not time sensitive and are not visible to the user.

backing store The secondary storage area used for process swapping.

backup In file systems, a copy or copies of the file system or changes to the file system that can be used to restore the file system if it is damaged or destroyed.

bad block An unusable sector on an HDD.

balanced binary search tree A tree containing n items that has, at most, $\lg n$ levels, thus ensuring worst-case performance of $O(\lg n)$.

bandwidth The total amount of data transferred divided by the total time between the first request for service and the completion of the last transfer.

banker's algorithm A deadlock avoidance algorithm, less efficient than the resource-allocation graph

scheme but able to deal with multiple instances of each resource type.

base file record In the Windows NTFS file system, a descriptor of a large file containing pointers to overflow records that hold additional pointers and attributes.

base register A CPU register containing the starting address of an address space. Together with the limit register, it defines the logical address space.

basic file system A logical layer of the operating system responsible for issuing generic commands to the I/O control layer, such as "read block x ," and also buffering and caching I/O.

batch interface A method for giving commands to a computer in which commands are entered into files, and the files are executed, without any human interaction.

Bayes' theorem A formula for determining conditional probability—e.g., the probability of an intrusion record detecting a real intrusion.

Belady's anomaly An anomaly in frame-allocation algorithms in which a page-fault rate may increase as the number of allocated frames increases.

best-fit In memory allocation, selecting the smallest hole large enough to satisfy the memory request.

big data Extremely large sets of data; distributed systems are well suited to working with big data.

big.LITTLE ARM processor implementation of HMP in which high-performance big cores are combined with energy efficient LITTLE cores.

big-endian A system architecture in which the most significant byte in a sequence of bytes is stored first.

binary search tree A type of binary tree data structure that requires an ordering between the parent's two children in which left child $\leftarrow$ right child.

binary semaphore A semaphore of values 0 and 1 that limits access to one resource (acting similarly to a mutex lock).

binary translation A virtualization method in which, when a guest is running in virtual kernel mode, every instruction is inspected by the virtual machine manager, and instructions that would behave differently in real kernel mode are translated into a set of new instructions that perform the equivalent task; used to implement virtualization on systems lacking hardware support for virtualization.

binary tree A tree data structure in which a parent may have at most two children.

G-4 Glossary

binder In Android RPC, a framework (system component) for developing object-oriented OS services and allowing them to communicate.

bind Tie together. For example, a compiler binds a symbolic address to a relocatable address so the routine or variable can be found during execution.

Bionic A type of standard C library used by Android; it has a smaller memory footprint than glibc and is more efficient on slower (mobile) CPUs.

BIOS Code stored in firmware and run at boot time to start system operation.

bit The basic unit of computer storage. A bit can contain one of two values, 0 or 1.

bit vector A string of n binary digits that can be used to represent the status of n items. The availability of each item is indicated by the value of a binary digit: 0 means that the resource is available, while 1 indicates that it is unavailable (or vice-versa).

bit-level striping The splitting of data at the bit level, with each bit in a byte or word stored on a separate device.

bitmap A string of n binary digits that can be used to represent the status of n items. The availability of each item is indicated by the value of a binary digit: 0 means that the resource is available, while 1 indicates that it is unavailable (or vice-versa).

BKL In Linux kernel version 2.2, the “big kernel lock” spinlock, which protected all kernel data structures.

blade server A computer with multiple processor boards, I/O boards, and networking boards placed in the same chassis. The difference between these and traditional multiprocessor systems is that each blade-processor board boots independently and runs its own operating system.

block A self-contained unit of work. The smallest physical storage device storage unit, typically 512B or 4KB. In the Grand Central Dispatch Apple OS scheduler, a language extension that allows designation of a section of code that can be submitted to dispatch queues.

block cipher A cipher that works on blocks of data (rather than bits).

block device An I/O device that is randomly accessible, with block-size chunks being the smallest I/O unit.

block-device interface The interface for I/O to block devices.

blocking In interprocess communication, a mode of communication in which the sending process is

blocked until the message is received by the receiving process or by a mailbox and the receiver blocks until a message is available. In I/O, a request that does not return until the I/O completes.

block-level striping The splitting of data at the block level, with each block stored on a separate device.

boot block A block of code stored in a specific location on disk with the instructions to boot the kernel stored on that disk. The UFS boot control block.

boot control block A storage block of data containing information needed by the system to boot from the volume containing the block.

boot disk A disk that has a boot partition and a kernel to load to boot the system. A device that has a boot partition and can store an operating system for booting the computer.

boot partition A storage device partition containing an executable operating system.

boot sector The first sector of a Windows boot device, containing the bootstrap code.

booting The procedure of starting a computer by loading the kernel.

bootstrap The set of steps taken at computer power-on to bring the system to full operation.

bootstrap loader The small program that loads the kernel as part of the bootstrap procedure.

bootstrap port In Mach message passing, a predefined port that allows a task to register a port it has created.

bootstrap program The program that allows the computer to start running by initializing hardware and loading the kernel.

bootstrap server In Mach message passing, a system-wide service for registering ports.

Border Gateway Protocol A network protocol for determining network routes so that packets can move from source to destination across a WAN.

bottleneck A performance-limiting aspect of computing (e.g., poorly written code or a hardware component that is not as fast as others in the system).

bounded buffer A buffer with a fixed size.

bounded waiting A practice that places limits on the time a thread or process is forced to wait for something.

Bourne-Again shell A common shell, or command interpreter.

BPF compiler collection (BCC) A rich toolkit for tracing system activity on Linux for debugging and performance-tuning purposes.

bridging In networking, the connection of two devices—e.g., a virtual-machine guest and the network to which the host computer is connected.

broadcast In networking, the sending of one or more packets to all devices on the local network.

browser A process that accepts input in the form of a URL (Uniform Resource Locator), or web address, and displays its contents on a screen.

buddies Pairs of equal size, used in the buddy method of memory allocation.

buddy heap In Linux, the use of a power-of-two allocator for the kernel heap.

buffer A memory area that stores data being transferred (e.g., between two devices or between a device and a process).

buffer cache In file I/O, a cache of blocks used to decrease device I/O.

bug An error in computer software or hardware.

bus A communication system; e.g., within a computer, a bus connects various components, such as the CPU and I/O devices, allowing them to transfer data and commands.

busy waiting A practice that allows a thread or process to use CPU time continuously while waiting for something. An I/O loop in which an I/O thread continuously reads status information while waiting for I/O to complete.

byte Eight bits.

bytecode A computer object code resulting from compiling a program in a language (such as Java) that runs on a virtual machine.

C library (libc) The standard UNIX/Linux system API for programs written in the C programming language.

cache A temporary copy of data stored in a reserved memory area to improve performance. In the slab allocator, a cache consists of two or more slabs.

cache coherency The coordination of the contents of caches such that an update to a value stored in one cache is immediately reflected in all other caches that hold that value.

cache management The management of a cache's contents.

cache-consistency problem A challenge in caching in which the cached copies of data must be kept consistent with the master copy of the data.

caching The use of temporary data storage areas to improve performance.

cancellation point With deferred thread cancellation, a point in the code at which it is safe to terminate the thread.

Canonical A popular Linux distribution.

capability In protection, the representation of an object in a capability list.

capability list In protection, a list of objects together with the operations allowed on those objects.

capability-based protection A protection facility in which the powers of root are divided into specific abilities, each represented by a bit in a bit mask that is used to allow or deny operations.

cascading termination A technique in which, when a process is ended, all of its children are ended as well.

Ceph A brand of object storage management software.

certificate authority A trusted signer of digital certificates.

character device An I/O device that has a character (byte) as its smallest I/O unit.

character-stream interface The interface for I/O to character devices (like keyboards).

checksum The general term for an error detection and correction code.

children In a tree data structure, nodes connected below another node.

chip multithreading (CMT) A CPU with multiple cores, where each core supports multiple hardware threads.

chipset The CPU and support chips that create a computer and define its architecture.

circular buffer A buffer that uses a fixed amount of storage and wraps writes from the end of the buffer to the beginning when the buffer fills (so that the buffer acts as if it's circular in shape).

Circular SCAN (CSCAN) scheduling An HDD I/O scheduling algorithm in which the disk head moves from one end of the disk to the other performing I/O as the head passes the desired cylinders; the head then reverses direction and continues.

claim edge In the deadlock resource-allocation-graph algorithm, an edge indicating that a process might claim a resource in the future.

class loader In Java, a helper component that loads .class files for execution by the Java virtual machine.

class In Java, a program component that is a collection of data fields and functions (methods) that operate on those fields.

clean-up handler A function that allows any resources a thread has acquired to be released before the thread is terminated.

client A computer that uses services from other computers (such as a web client). The source of a communication.

client interface In distributed computing, a set of services provided to a caller of services.

client system A computer that uses services from other computers (such as a web client).

client-server model A mode of computing in which a server provides services to one or more clients. In distributed computing, a model in which a computer acts as a resource server to other computers that are clients of those resources.

client-side caching (CSC) In Windows, a caching method used to allow remote users to work off-line and then consolidate their changes once they are online.

clock In the second-chance page-replacement algorithm, a circular queue that contains possible victim frames. A frame is replaced only if it has not been recently referenced.

clone In file systems, a snapshot that is read-write and modifiable. In virtualization, a copy of a guest that enables another instance of the guest to run in a separate virtual machine.

CLOOK scheduling An HDD I/O scheduling algorithm that modifies CSCAN by stopping the head after the final request is completed (rather than at the innermost or outermost cylinder).

closed-source An operating system or other program available only in compiled binary code format.

closure In functional programming languages, a construct to provide a simple syntax for parallel applications.

cloud computing A computing environment in which hardware, software, or other resources are made available to customers across a WAN, such as the Internet, usually with APIs for management. A type of computing that delivers computing, storage, and even applications “as a service” across a network.

cloud storage Storage accessed from a computer over a network to a distant, shared resource data center.

cluster In Windows storage, a power-of-2 number of disk sectors collected for I/O optimization.

cluster-based model In distributed computing, a model of resource sharing where all systems are equal users and providers of resources.

clustered file system (CFS) A file system that is LAN-based and treats N systems storing data and Y clients accessing the data as a single client-server

instance; more complex than a client-server DFS but less complex than a cluster-based DFS. GPFS and Lustre are examples.

clustered page table A page table similar to a hashed page table, but a table entry refers to several (a cluster of) pages.

clustered system A system that gathers together multiple CPUs. Clustered systems differ from multiprocessor systems in that they are composed of two or more individual systems—or nodes—joined together.

clustering In general, gathering N items together. In virtual memory, paging in a group of contiguous pages when a single page is requested via a page fault.

cluster In file system block allocation, several contiguous blocks.

coalescing In general, combining. In the buddy memory allocation algorithm, freed memory in adjacent buddies can be coalesced into larger segments.

code integrity A Windows 7 module that checks the digital signatures of kernel modules to be sure they have not been tampered with by attackers.

code review A software development method in which the developer submits code to other developers for review and approval.

code signing The use of a digital signature to authenticate a program.

code-injection attack An attack that modifies otherwise well-behaved executable code.

command interpreter The operating system component that interprets user commands and causes actions based on them.

command-line interface (CLI) A method of giving commands to a computer based on a text input device (such as a keyboard).

Common Criteria The international 2005 successor to the Orange Book standard developed by the U.S. Department of Defense.

common Internet file system (CIFS) The Windows network file system, now used on many systems.

communication port In Windows OS, a port used to send messages between two processes.

communications A category of system calls.

compaction The shuffling of storage to consolidate used space and leave one or more large holes available for more efficient allocation.

compartmentalization The process of protecting each system component through the use of specific permissions and access restrictions.

Completely Fair Queuing (CFQ) In Linux, the default I/O scheduler in kernel 2.6 and later versions.

Completely Fair Scheduler (CFS) In Linux, the priority-based, preemptive scheduler included with the 2.6 kernel.

component object model (COM) The Windows mechanism for interprocess communication.

compression The use of algorithms to reduce the amount of data stored or sent.

compression ratio In memory compression, a measurement of the effectiveness of the compression (the ratio of the compressed space to the original amount of uncompressed space).

compression unit In NTFS, a unit of 16 contiguous clusters used in memory compression.

computation migration The use of a network to allow a task to remotely request resources to speed a computation.

computation speedup An increase in the amount of CPU compute power.

computational kernel A Windows mechanism for specifying tasks to run on GPUs.

compute-server system A server that provides an interface to which a client can send a request for an action (e.g., read data). In response, the server executes the action and sends the results to the client.

Concurrency Runtime (ConcRT) A Microsoft Windows concurrent programming framework for C++ that is designed for task-based parallelism on multicore processors.

condition variable A component of a monitor lock; a container of threads waiting for a condition to be true to enter the critical section.

conditional-wait A component of the monitor construct that allows for waiting on a variable with a priority number to indicate which process should get the lock next.

confinement problem The problem of guaranteeing that no information initially held in an object can migrate outside of its execution environment.

conflict phase During scheduling, the time the dispatcher spends moving a thread off a CPU and releasing resources held by lower-priority threads that are needed by the higher-priority thread that is about to be put onto the CPU.

conflict-resolution mechanism In Linux, the kernel module facility that allows different device drivers to reserve hardware resources and to protect those resources from accidental use by other drivers.

congestion control In networking, the attempt to approximate the state of networks between senders and receivers to avoid packet loss.

connection port In Windows OS, a communications port used to maintain connection between two processes, published by a server process.

connectionless socket (UDP) In Java, a mode of communication.

connection-oriented socket (TCP) In Java, a mode of communication.

consistency checker A system program, such as UNIX fsck, that reads file system metadata and tries to correct any inconsistencies (such as a file's length being incorrectly recorded).

consolidation In virtualization, the practice of running multiple guests per host, reducing the number of physical servers needed for a given workload.

constant angular velocity (CAV) A device-recording method in which the medium spins at a constant velocity and the bit density decreases from inner to outer tracks.

constant linear velocity (CLV) A device-recording method that keeps a constant density of bits per track by varying the rotational speed of the medium.

consumer A process role in which the process consumes information produced by a producer process.

container In application containment, a virtual layer between the operating system and a process in which the application runs, limiting its normal access to system resources.

container object In Windows 10, a category of objects that contain other objects (e.g., directories, which contain files).

containers In APFS, large free spaces in storage from which file systems can draw allocations.

containment A method for preventing deadlocks by creation and lock management of a higher-order resource that contains the locks of lower-order resources.

contended A term describing the condition of a lock when a thread blocks while trying to acquire it.

content addressable storage Another term for object storage; so called because objects can be retrieved based on their contents.

context When describing a process, the state of its execution, including the contents of registers, its program counter, and its memory context, including its stack and heap.

context switch The switching of the CPU from one process or thread to another; requires performing a state save of the current process or thread and a state restore of the other.

G-8 Glossary

contiguous allocation A file-block allocation method in which all blocks of the file are allocated as a contiguous chunk on secondary storage.

contiguous bit In ARM v8 CPUs, a TLB bit indicating that the TLB holds a mapping to contiguous blocks of memory.

contiguous memory allocation A memory allocation method in which each process is contained in a single section of memory that is contiguous to the section containing the next process.

control partition In type 0 virtualization, a virtual hardware partition that provides services to the other partitions (and has higher privileges).

control program A program that manages the execution of user programs to prevent errors and improper use of the computer. It is especially concerned with the operation and control of I/O devices.

control register A device I/O register where commands are placed by the computer.

control-flow guard (CFG) A Windows 7 exploit-mitigation feature.

controller A special processor that manages I/O devices.

convoy effect A scheduling phenomenon in which a number of threads wait for one thread to get off a core, causing overall device and CPU utilization to be suboptimal.

cooperating process A process that can affect or be affected by other processes executing in the system.

cooperative A form of scheduling in which threads voluntarily move from the running state.

coordination Ordering of the access to data by multiple threads or processes.

copy protection The implementation of a mechanism to prevent the unauthorized copying of a licensed work (e.g., media, programs, operating systems).

copy semantics The meaning assigned to data copying—e.g., whether a block write from a process allows the data to be modified after the write has been requested.

copy-on-write Generally, the practice by which any write causes the data to first be copied and then modified, rather than overwritten. In virtual memory, on a write attempt to a shared page, the page is first copied, and the write is made to that copy.

core Within a CPU, the component that executes instructions.

core dump A copy of the state of a process written to disk when a process crashes; used for debugging.

core frameworks In the layered macOS and iOS operating system design, the layer that defines frameworks that support graphics and media, including QuickTime and OpenGL.

counting semaphore A semaphore that has a value between 0 and N, to control access to a resource with N instances.

CPU burst Scheduling process state in which the process executes on CPU.

CPU scheduler Kernel routine that selects a thread from the threads that are ready to execute and allocates a core to that thread.

CPU scheduling The process by which the system chooses which job will run next. If several jobs are ready to run at the same time.

CPU-bound process A process that spends more time executing on CPU than it does performing I/O.

crash Termination of execution due to a problem. A failure in the kernel results in a system crash and reboot, while a process failure results in a process crash.

crash dump A copy of the state of the kernel written to disk during a crash; used for debugging.

critical section A section of code responsible for changing data that must only be executed by one thread or process at a time to avoid a race condition.

critical-section object A user-mode mutex object that can often be acquired and released without kernel intervention; a Windows OS scheduling feature.

cryptography A tool used to constrain the potential senders and/or receivers of a message (or stored data).

current-file-position pointer A per-process pointer to the location in a file to which the next read or from which the next write will occur.

cycle A repeating loop.

cycle stealing The act of a device, such as a DMA controller, using the bus and preventing the CPU from using it temporarily.

cylinder On an HDD, the set of tracks under the read-write heads on all platters in the device.

cylinder groups A sequential collection of storage media (typically a group of cylinders) that are managed as a single unit and subdivided into blocks.

daemon A service that is provided outside of the kernel by system programs that are loaded into memory at boot time and run continuously.

daisy chain In computer I/O, a connection method involving connecting devices to each other in a string (device A to B, B to C, C to D, etc.).

dark web The part of the World Wide Web that is not easy to reach (say, by search engines) and that is sometimes used for bad behavior (such as selling information stolen in successful attacks).

Darwin The Apple code name for its open-source kernel.

data execution prevention (DEP) A Windows 7 exploit-mitigation feature.

data migration The entire transfer of one or more files from one site to another.

data parallelism A computing method that distributes subsets of the same data across multiple cores and performs the same operation on each core.

data section The data part of a program or process; it contains global variables.

data striping The splitting of data across multiple devices.

data-encryption standard (DES) A cipher (algorithm for doing encryption and decryption) provided by the U.S. National Institute of Standards and Technology (NIST).

datagram A basic transfer unit on a packet-switched network like the Internet protocol (i.e., a network packet).

data-in register A device I/O register where data is placed to be sent to the device.

data-out register A device I/O register where data is placed by the device to be read by the computer.

DCOM The distributed computing extension to object linking and embedding (OLE).

deadlock The state in which two processes or threads are stuck waiting for an event that can only be caused by one of the processes or threads.

deadlock avoidance An operating system method in which processes inform the operating system of which resources they will use during their lifetimes so the system can approve or deny requests to avoid deadlock.

deadlock prevention A set of methods intended to ensure that at least one of the necessary conditions for deadlock cannot hold.

deadlocked The state in which two processes or threads are stuck waiting for an event that can only be caused by one of the processes or threads.

Debian A popular Linux distribution.

debugger A system program designed to aid programmers in finding and correcting errors.

debugging The activity of finding and removing errors.

deduplication The removal of duplicate information (bits, blocks, or files).

default heap The heap data structure created when a Win32 process is initialized.

default signal handler The signal handler that receives signals unless a user-defined signal handler is provided by a process.

defense in depth The theory that more layers of defense provide stronger defense than fewer layers.

deferred procedure call (DPC) In Windows scheduling, a call initiated by the interrupt that occurs when a time quantum expires, eventually causing the expired thread to be moved off a core and replaced with the next thread in the ready queue.

degree of multiprogramming The number of processes in memory.

delayed-write policy In caching, a policy whereby data are first written to a cache; later, the cache writes the change to the master copy of the data.

demand paging In memory management, bringing in pages from storage as needed rather than, e.g., in their entirety at process load time.

demand-zero memory In Linux, a virtual memory region backed by nothing; when a process tries to read a page in such a region, it is given a page of memory filled with zeros.

demilitarized zone In firewalling, a security domain less trusted than some other security domain (e.g., the domain containing a web server compared to the domain containing the crucial company database).

denial-of-service Preventing legitimate use of a system.

dentry object The VFS representation of a directory.

desktop In a GUI, the standard workspace represented by the GUI on the screen, in which a user executes tasks.

desktop activity moderator (DAM) A Windows 8 component that supports the system state “connected standby” on mobile devices, freezing computer activity but allowing rapid return to full functionality.

desktop window manager A Windows Vista user-mode component to manage GUI windows.

deterministic modeling A type of analytic evaluation that takes a particular predetermined workload and defines the performance of each algorithm for that workload.

development kernel A kernel released for developers to use, rather than for production use.

device controller The I/O managing processor within a device.

G-10 Glossary

device driver An operating system component that provides uniform access to various devices and manages I/O to those devices.

device manipulation A category of system calls.

device object In Windows, the object representing a device.

device queue The list of processes waiting for a particular I/O device.

device-status table A kernel data structure for tracking the status and queues of operations for devices.

digital certificate A public key digitally signed by a trusted party.

digital rights management The implementation of a mechanism to prevent the unauthorized copying of a licensed work (e.g., media, programs, operating systems).

digital signature The authenticator produced by a digital-signature algorithm.

digital-signature algorithm A cryptographic checksum calculated in asymmetric encryption; used to authenticate a message.

dining-philosophers problem A classic synchronization problem in which multiple operators (philosophers) try to access multiple items (chopsticks) simultaneously.

direct access A file-access method in which contents are read in random order, or at least not sequentially.

direct blocks In UFS, blocks containing pointers to data blocks.

direct communication In interprocess communication, a communication mode in which each process that wants to communicate must explicitly name the recipient or sender of the communication.

direct I/O Block I/O that bypasses operating-system block features such as buffering and locking.

direct memory access (DMA) A resource-conserving and performance-improving operation for device controllers allowing devices to transfer large amounts of data directly to and from main memory.

direct virtual memory access (DVMA) DMA that uses virtual addresses rather than physical memory addresses as transfer sources and destinations.

dirty bit An MMU bit used to indicate that a frame has been modified (and therefore must have its contents saved before page replacement).

discretionary access control (DAC) Optional, as opposed to mandatory, access control.

disinfecting In the context of computer viruses, removing the components of a virus.

disk arm An HDD component that holds the read-write head and moves over cylinders of platters.

disk image Generally, the contents of a disk encapsulated in a file. In virtualization, a guest virtual machine's boot file system contents contained in a disk image.

diskless A term describing systems that have no local storage.

dispatch latency The amount of time the dispatcher takes to stop one thread and put another thread onto the CPU.

dispatch queue An Apple OS feature for parallelizing code; blocks are placed in the queue by Grand Central Dispatcher (GCD) and removed to be run by an available thread.

dispatched Selected by the process scheduler to be executed next.

dispatcher The kernel routine that gives control of a core to the thread selected by the scheduler.

dispatcher objects A Windows scheduler feature that controls dispatching and synchronization. Threads synchronize according to several different mechanisms, including mutex locks, semaphores, events, and timers.

distributed denial-of-service attack (DDoS) An attack from multiple sources (frequently a botnet of zombies) with the purpose of denying legitimate use of the attacked resource.

distributed file system (DFS) A file system that works across a network in which remote directories are visible from a local machine.

distributed information system A set of protocols providing unified information needed for remote computing.

distributed lock manager (DLM) A function used by a clustered system to supply access control and locking to ensure that no conflicting operations occur.

distributed naming service A set of protocols providing unified information needed for remote computing.

distributed system A collection of loosely coupled nodes interconnected by a communication network.

distribution A release of a version of an operating system.

docker An orchestration tool for containers.

domain switching The mechanism for switching dynamic domains.

domain-name system (DNS) The Internet system for resolving host names to host-ids, in which

a distributed information system provides host-name-to-network-address translations for the Internet.

double buffering The copying of data twice (e.g., from a device to the kernel and then from the kernel to a process's address space), or the use of two buffers to decouple producers and consumers.

double caching The problem in which the same data might be in two different caches; solved by a unified buffer cache.

double indirect block In UFS, a block containing pointers to a single indirect block, which points to data blocks.

down time Generally, time during which a facility, system, or computing component are off-line and unavailable.

driver end The interface between STREAMS and the device being controlled.

driver object In Windows, the object representing a device driver.

driver-registration system In Linux, the kernel module facility that tells the rest of the kernel that a new driver is available.

DTrace A facility originally developed at Sun Microsystems that dynamically adds probes to a running system, both in user processes and in the kernel, for state analysis, performance tuning, and debugging.

dual-booted A term describing a computer that can boot one of two or more installed operating systems.

dynamic loading The loading of a process routine when it is called rather than when the process is started.

dynamic random-access memory (DRAM) The common version of RAM, which features high read and write speeds.

dynamic storage-allocation The class of file-block allocation methods that satisfy a request for *n* blocks from a list of free holes available.

dynamic storage-allocation problem The problem of how to satisfy a request for size *n* of memory from a list of free holes.

dynamically linked libraries (DLLs) System libraries that are linked to user programs when the processes are run, with linking postponed until execution time.

earliest-deadline-first (EDF) A real-time scheduling algorithm in which the scheduler dynamically assigns priorities according to completion deadlines.

ease of use The amount of difficulty and complexity involved in using some aspect of computing.

EEPROM Electrically erasable programmable read-only memory; a type of firmware.

effective access time The measured or statistically calculated time it takes to access something; e.g., see effective memory-access time.

effective memory-access time The statistical or real measure of how long it takes the CPU to read or write to memory.

effective transfer rate The actual, measured transfer rate of data between two devices (such as a computer and a disk drive).

effective UID The UID the process is currently using, which can be different from the login UID due to, e.g., escalating privileges.

electrical storage Storage devices made from electrical components, such as flash memory; one form of nonvolatile storage.

ELF The UNIX standard format for relocatable and executable files.

embedded computer A computer system within some other, larger system (such as a car) that performs specific, limited functions and has little or no user interface.

emulation A methodology used to enable a process to run when the compiled program's original (source) CPU type is different from the target CPU type on which the program is to run.

emulator A program that allows applications written for one hardware environment to run in a different hardware environment, such as a different type of CPU.

encapsulate In general, to enclose. In Java, encapsulation gives a class the ability to protect its data and methods from other classes loaded in the same JVM.

encryption The use of cryptography to limit the receivers of a message or access to data.

entry section The section of code within a process that requests permission to enter its critical section.

entry set In Java, the set of threads waiting to enter a monitor.

environment vector In Linux and UNIX, a list containing all of the environment variables of the shell that invoked a process (and are available to the process).

environmental subsystem In Windows, an operating environment that emulates a specific operating system by translating process API calls from that operating system to Windows calls.

G-12 Glossary

equal allocation An allocation algorithm that assigns equal amounts of a resource to all requestors. In virtual memory, assigning an equal number of frames to each process.

error-correcting code (ECC) A value calculated from bytes of data and recalculated later to check for changes in the data.

eSATA A type of I/O bus.

escalate privileges To gain extra permissions for an activity, as when a user needs access to a device that is restricted.

escape Generally, a method of passing arbitrary commands or information when an interface does not provide a standard method.

event latency The amount of time between when an event occurs and when it is serviced.

event A Windows OS scheduling feature that is similar to a condition variable.

eVM An example of a partitioning hypervisor.

exception A software-generated interrupt caused either by an error (such as division by zero or invalid memory access) or by a specific request from a user program than an operating-system service be performed.

exception dispatcher The Windows component that processes exceptions.

exclusive lock A file lock similar to a writer lock in that only one process at a time can obtain the lock.

executable and linkable format (ELF) The UNIX standard format for relocatable and executable files.

executable file A file containing a program that is ready to be loaded into memory and executed.

execution tracing A Windows method of monitoring process execution and recording details for analysis and debugging.

exit section The section of code within a process that cleanly exits the critical section.

expansion bus A computer bus for connecting slow devices like keyboards.

exponential average A calculation used in scheduling to estimate the next CPU burst time based on the previous burst times (with exponential decay on older values).

export list The list of file systems available for remote mounting from a server.

ext3 One of the most common types of Linux file systems.

extended file attributes Extended metadata about a file, including items such as character encoding details, file checksums, etc.

extended file system The most common class of Linux file systems, with ext3 and ext4 being the most commonly used file system types.

extended file system (extfs) The most common class of Linux file systems, with ext3 and ext4 being the most commonly used file system types.

extensibility The ability of an operating system to accommodate advances in computing technology via extensions (such as new kernel modules and new device drivers).

extent A chunk of contiguous storage space added to previously allocated space; a pointer is used to link the spaces.

external data representation A system used to resolve differences when data are exchanged between big- and little-endian systems.

external fragmentation Fragmentation in which available memory contains holes that together have enough free space to satisfy a request but no single hole is large enough to satisfy the request. More generally, the fragmentation of an address space into small, less usable chunks.

fair scheduling In the Completely Fair Scheduler, the scheduler algorithm that allots a proportion of the processor's time rather than time slices.

false negatives Results indicating that something is not a match to what is being detected, even though it is.

false positives Results indicating that something is a match to what is being detected, even though it isn't.

fast directory sizing In APFS, a feature that tracks and rapidly reports on directory sizes.

fast I/O mechanism In Windows, a high-speed bypass of the standard I/O handling mechanism in which the driver stack is called directly rather than having an IRP sent and processed.

fault-tolerant system A system that can suffer a failure of any single component and still continue operation.

fiber User-mode code that can be scheduled according to a user-defined scheduling algorithm.

fibre channel (FC) A type of storage I/O bus used in data centers to connect computers to storage arrays. A storage-attachment network.

file The smallest logical storage unit; a collection of related information defined by its creator.

file attributes Metadata about a file, such as who created it and when, file size, etc.

file descriptor (fd) UNIX open-file pointer, created and returned to a process when it opens a file.

file handle Windows name for the open-file file descriptor.

file info window A GUI view of the file metadata.

file manipulation A category of system calls.

file mapping In Windows, the first step in memory-mapping a file.

file migration A file system feature in which a file's location is changed automatically by the system.

file object The VFS representation of an open file.

file reference In Windows NTFS, a unique file ID that is the index of the file in the master file table (much like a UNIX inode number).

file session The series of accesses between open() and close() system calls that make up all the operations on a file by a process.

file system The system used to control data storage and retrieval; provides efficient and convenient access to storage devices by allowing data to be stored, located, and retrieved easily.

File System Hierarchy Standard A standard document specifying the overall layout of the standard Linux file systems.

file-allocation table (FAT) A simple but effective method of disk-space allocation used by MS-DOS. A section of storage at the beginning of each volume is set aside to contain the table, which has one entry per block, is indexed by block number, and contains the number of the next block in the file.

file-control block (FCB) A per-file block that contains all the metadata about a file, such as its access permissions, access dates, and block locations.

file-organization module A logical layer of the operating system responsible for files and for translation of logical blocks to physical blocks.

file-server system A server that provides a file-system interface where clients can create, update, read, and delete files (e.g., a web server that delivers files to clients running web browsers).

filter drivers In Windows, drivers allowed to insert themselves into the I/O processing chain.

firewall A computer, appliance, process, or network router that sits between trusted and untrusted systems or devices. It protects a network from security breaches by managing and blocking certain types of communications.

firewall chains In Linux, an ordered list of rules that specifies one of a number of possible firewall-decision functions plus matching components. A configuration of the firewall rules.

firmware Software stored in ROM or EEPROM for booting the system and managing low level hardware.

first-come first-served (FCFS) The simplest scheduling algorithm. The thread that requests a core first is allocated the core first.

first-fit In memory allocation, selecting the first hole large enough to satisfy a memory request.

first-level interrupt handler In some operating systems, an interrupt handler responsible for reception and queuing of interrupts; the interrupts are actually handled at another level (by the second-level handler).

flash translation layer (FTL) For nonvolatile memory, a table that tracks currently valid blocks.

flow control Generally, a method to pause a sender of I/O. In networking, a technique to limit the rate of data flow (e.g., to avoid buffer overflow and packet loss on a router).

flush Erasure of entries in, e.g., a TLB or other cache to remove invalid data.

folder A file system component that allows users to group files together.

folder redirection In Windows, for roaming users, a method for automatically storing a user's documents and other files on a remote server.

foreground Describes a process or thread that is interactive (has input directed to it), such as a window currently selected as active or a terminal window currently selected to receive input.

foreground process A process currently open and appearing on the display, with keyboard or other I/O device input directed to it.

fork-join A strategy for thread creation in which the main parent thread creates (forks) one or more child threads and then waits for the children to terminate and join with it.

forward-mapped Describes a scheme for hierarchical page tables in which address translation starts at the outer page table and moves inward.

fourth extended file system (ext4) In Linux, a current version of the extended file system, the successor to ext3.

fragments In networking, parts of messages. A message is split into fragments if it is too large to fit in one packet.

frame table In paged memory, the table containing frame details, including which frames are allocated, which are free, total frames in the system, etc.

frame-allocation algorithm The operating-system algorithm for allocating frames among all demands for frames.

frames Fixed-sized blocks of physical memory.

G-14 Glossary

free operating system An operating system released under a license that makes its source code available and allows no-cost use, redistribution, and modification.

free-behind Sequential I/O performance optimization that removes a page or block from a buffer as soon as I/O to the next page is requested.

free-frame list A kernel-maintained data structure containing the list of currently available free frames of physical memory.

free-space list In file-system block allocation, the data structure tracking all free blocks in the file system.

front-end processors Small computers that perform certain tasks in an overall system; used by some systems to manage I/O and offload the general-purpose CPU.

fsgid In Linux, an added process property that allows file system access on behalf of another group.

fsuid In Linux, an added process property that allows file system access on behalf of another user.

full backup In file systems, a backup that includes all of the contents of a file system.

functional language A programming language that does not require states to be managed by programs written in that language (e.g., Erlang and Scala).

fuzzing A technique that tests for proper input validation (e.g., to avoid undetected buffer overruns).

Galois field math An advanced error-correcting calculation done in some forms of RAID.

Gantt chart A bar chart that can be used to illustrate a schedule.

garbage collection In general, recovery of space containing no-longer-valid data.

general tree A tree data structure in which a parent may have unlimited children.

gestures A user interface component in which motions cause computer actions (e.g., “pinching” the screen).

gigabyte (GB) $1,024^3$ bytes.

git A version control system used for GNU/Linux and other programs.

global replacement In virtual memory frame allocation, allowing a process to select a replacement frame from the set of all frames in the system, not just those allocated to the process.

GNU C compiler (gcc) The standard C compiler of the GNU project, used throughout the industry on Linux and many other system.

GNU General Public License (GPL) A license agreement that codifies copylefting (allowing and requiring open sourcing of the associated programs); a common license under which free software is released.

GNU/Linux (aka Linux) An open-source operating system composed of components contributed by the GNU foundation and Linus Torvalds, as well as many others.

Google Android The mobile operating system created by Google Inc.

Google file system (GFS) A cluster-based distributed file system designed and used by Google.

GPFS A common commercial clustered file system by IBM.

graceful degradation The ability of a system to continue providing service proportional to the level of surviving hardware.

graphical user interface (GUI) A computer interface comprising a window system with a pointing device to direct I/O, choose from menus, and make selections and, usually, a keyboard to enter text.

graphics processing unit (GPU) A hardware component, sometimes part of the CPU and sometimes a separate device, that provides graphics computation and sometimes graphics output to a display.

graphics shaders Processes that provide the shading within graphics images.

group In file permissions, a collection of users that have access to a file based on assigned access rights.

group identifier Similar to a user identifier, but used to identify a group of users to determine access rights.

group rights In file permissions, the access rights belonging to a user group.

GRUB A common open-source bootstrap loader that allows selection of boot partitions and options to be passed to the selected kernel.

guard pages In Windows, no-access-allowed pages at the tops of the kernel-mode and user-mode stacks that detect stack overflows.

guest In virtualization, an operating system running in a virtual environment (rather than natively on the computer hardware).

hacker Someone attempting to breach computer security.

Hadoop distributed file system (HDFS) A cluster-based distributed file system used for big-data projects.

Hadoop file system An example of object storage management software.

haiku A three-line poem in which the first line contains five syllables, the second line contains seven syllables, and the third line contains five syllables.

handle Generally, an opaque value that provides access to an object or data structure; e.g., a file handle is returned when a file is opened and is a pointer to an entry in an open-file table.

handle table In Windows, a per-process handle table containing entries that track (by their handles) the objects used by the process.

hard affinity The situation in which an operating system allows a process's threads to run on the same processor at all times (as opposed to being moved to various processors).

hard disk drive (HDD) A secondary storage device based on mechanical components, including spinning magnetic media platters and moving read-write heads.

hard error An unrecoverable error (possibly resulting in data loss).

hard links File-system links in which a file has two or more names pointing to the same inode.

hard real-time systems Systems in which a thread must be serviced by its deadline; service after the deadline has expired is the same as no service at all.

hard working-set limit In Windows memory management, the maximum amount of physical memory that a process is allowed to use.

hardware The CPU, memory devices, input/output (I/O) devices, and any other physical components that are part of a computer.

hardware objects The CPU, memory devices, input/output (I/O) devices, and any other physical components that are part of a computer

hardware threads Threads associated with the processing core. A given CPU core may run multiple hardware threads to optimize core use—e.g., to avoid memory stalls by switching hardware threads if the current thread causes a stall.

hardware transactional memory (HTM) A transactional memory implementation using hardware cache hierarchies and cache coherency protocols to manage and resolve conflicts involving shared data residing in separate processors' caches.

hardware-abstraction layer (HAL) A kernel layer that isolates chipset-specific hardware aspects from general-purpose code.

hash function A function that takes data as its input, performs a numeric operation on the data,

and returns a numeric value. Also, an algorithm for creating a hash (a small, fixed-size block of data calculated from a larger data set, used to determine if a message has been changed).

hash map A data structure that maps [key:value] pairs using a hash function; a hash function can then be applied to a key to obtain its matching value.

hash value The calculation resulting from a hash function.

hashed page table A page table that is hashed for faster access; the hash value is the virtual page number.

head crash On an HDD, a mechanical problem involving the read-write head touching a platter.

heap section The section of process memory that is dynamically allocated during process run time; it stores temporary variables.

heartbeat In computer clustering, a repeating signal to determine the state of the members of the cluster (e.g., to determine if a system is down).

heterogeneous multiprocessing (HMP) A feature of some mobile computing CPUs in which cores vary in their clock speeds and power management.

high-availability Describes a service that will continue even if one or more systems in the cluster fail.

high-performance computing A computing facility designed for use with a large number of resources to be used by software designed for parallel operation.

high-performance event timer A hardware timer provided by some CPUs.

hit ratio The percentage of times a cache provides a valid lookup (used, e.g., as a measure of a TLB's effectiveness).

hives In Windows, an internal repository of data.

hole In variable partition memory allocation, a contiguous section of unused memory. Also an alternative rock band formed by Courtney Love.

honeypot A false resource exposed to attackers; the resource appears real and enables the system to monitor and gain information about the attack.

horizontal scalability The ability to scale capacity not by expanding one item but by adding more items.

host In virtualization, the location of the virtual machine manager, which runs guest operating systems; generally, a computer.

host bus adapter (HBA) A device controller installed in a host bus port to allow connection of one or more devices to the host.

G-16 Glossary

host controller The I/O-managing processors within a computer (e.g., inside a host bus adapter).

host name A human-readable name for a computer.

host-attached storage Storage accessed through local I/O ports (directly attached to a computer, rather than across a network or SAN).

host-id In networking, the unique number identifying a system on a network.

hostname The alphanumeric name for a system (such as becca.colby.edu).

hot spare An unused storage device ready to be used to recover data (e.g., in a RAID set).

hot-standby mode A condition in which a computer in a cluster does nothing but monitor the active server. If that server fails, the hot-standby host becomes the active server.

huge pages A feature that designates a region of physical memory where especially large pages can be used.

hybrid cloud A type of cloud computing that includes both public and private cloud components.

hypercall In paravirtualization, a call from a guest to the hypervisor to request a virtualization service, such as a page table change.

hyper-threading Intel's technology for assigning multiple hardware threads to a single processing core.

hypervisor The computer function that manages the virtual machine; also called a virtual machine manager (VMM).

I/O burst Scheduling process state in which the CPU performs I/O.

I/O bus A physical connection of an I/O device to a computer system.

I/O channel A dedicated, special-purpose CPU found in large systems like mainframes for performing I/O or offloading the general-purpose CPU.

I/O control A logical layer of the operating system responsible for controlling I/O, consisting of device drivers and interrupt handlers.

I/O manager In Windows, the system component responsible for I/O.

I/O port A hardware connector allowing connection of an I/O device.

I/O request packet (IRP) In Windows, a data structure to request file I/O that is sent from the I/O manager to the appropriate device driver.

I/O subsystem The I/O devices and the part of the kernel that manages I/O.

I/O-bound process A process that spends more of its time doing I/O than doing computations

icons Images representing objects (such as files or applications) that users can choose via the GUI.

idempotent Describes a function that, when applied more than once, has the same result every time.

identifier Generally, a numeric tag for a device or object. In networking, the unique host-id number identifying a system on a network.

idle process In Windows, a process that serves as the container of all idle threads.

idle thread In some operating systems, a special thread that runs on the CPU when no other thread is ready to run.

immutable shared file In a remote file system, a file that, once shared by its creator, cannot be modified.

imperative language Language for implementing algorithms that are state-based (e.g., C, C++, Java, and C#).

impersonation In Windows, the representation of a thread by a token for security purposes.

implicit threading A programming model that transfers the creation and management of threading from application developers to compilers and run-time libraries.

incremental backup In file systems, a backup that contains only some parts of a file system (the parts that have changed since the last full and incremental backups).

indefinite blocking A situation in which one or more processes or threads waits indefinitely within a semaphore.

index In file systems, an access method built on top of direct access in which a file contains an index with pointers to the contents of the file.

index block In indexed allocation, a block that contains pointers to the blocks containing the file's data.

index root In NTFS, the part of the directory containing the top level of the B+ tree.

indexed allocation In file-system block allocation, combining all block pointers in one or more index blocks to address limits in linked allocation and allow direct access to each block.

indirect block In UFS, a block containing pointers to direct blocks, which point to data blocks.

InfiniBand (IB) A high-speed network communications link.

infinite blocking A scheduling risk in which a thread that is ready to run is never put onto the

CPU due to the scheduling algorithm; it is starved for CPU time.

information maintenance A category of system calls.

infrastructure as a service (IaaS) A type of computing in which servers or storage are available over the Internet (e.g. storage available for making backup copies of production data).

inode In many file systems, a per-file data structure holding most of the metadata of the file. The FCB in most UNIX file systems.

inode object The VFS representation of an individual file.

input/output operations per second A measure of random access I/O performance; the number of inputs + outputs per second.

integrity label In Windows Vista and later versions, a mandatory access control component assigned to each securable object and subject.

integrity levels A mechanism, introduced in Windows Vista, that acts as a rudimentary capability system for controlling access.

Intel 64 Intel 64 bit CPUs, part of a class of CPUs collectively known as x86-64

interactive Describes a type of computing that provides direct communication between the user and the system.

intermachine interface In distributed computing, a set of low-level functions for cross-machine interaction.

internal fragmentation Fragmentation that is internal to a partition.

Internet A worldwide system of interconnected computer networks.

Internet key exchange (IKE) A protocol that uses public key encryption to allow secure symmetric key exchange for IPsec.

Internet protocol (IP) The low-level networking protocol on which TCP and UDP are layered; responsible for routing IP datagrams through networks such as the Internet.

Internet protocol security (IPsec) A network protocol suite providing authentication and symmetric-key encryption of packets of network data.

Internet Service Providers (ISPs) Companies that provide Internet access.

interpretation A methodology that allows a program in computer language to be either executed in its high-level form or translated to an intermediate form rather than being compiled to native code.

interprocess communication (IPC) Communication between processes.

interrupt A hardware mechanism that enables a device to notify the CPU that it needs attention.

interrupt address A variable provided along with an interrupt signal to indicate the source of the interrupt.

interrupt chaining A mechanism by which each element in an interrupt vector points to the head of a list of interrupt handlers, which are called individually until one is found to service the interrupt request.

Interrupt latency The period of time from the arrival of an interrupt at the CPU to the start of the routine that services the interrupt.

interrupt object The Windows representation of an interrupt.

interrupt priority level Prioritization of interrupts to indicate handling order.

interrupt request level (IRQL) A prioritization method used in interrupt management.

interrupt service routine (ISR) An operating system routine that is called when an interrupt signal is received.

interrupt vector An operating-system data structure indexed by interrupt address and pointing to the interrupt handlers. A kernel memory data structure that holds the addresses of the interrupt service routines for the various devices.

interrupt-controller hardware Computer hardware components for interrupt management.

interrupt-dispatch table The Windows term for its interrupt vector.

interrupt-handler routine An operating system routine that is called when an interrupt signal is received.

interrupt-request line The hardware connection to the CPU on which interrupts are signaled.

intruder Someone attempting to breach security.

intrusion prevention The attempt to detect attempted and successful intrusions and properly respond to them.

intrusion-prevention systems (IPS) Systems to detect and prevent intrusions, usually in the form of self-modifying firewalls.

inverted page table A page-table scheme that has one entry for each real physical page frame in memory; the entry maps to a logical page (virtual address) value.

iSCSI The protocol used to communicate with SCSI devices; used across a network for more distant access.

G-18 Glossary

Itanium Intel IA-64 CPU.

iteration space In Intel threading building blocks, the range of elements that will be iterated.

Java virtual machine In programming-environment virtualization, the process that implements the Java language and allows execution of Java code.

job objects In Windows, data structures for tracking collections of processes (e.g., to set CPU usage limits).

job pool The location where jobs are kept on disk while waiting for main memory to become available.

job scheduling The task of choosing which jobs to load into memory and execute.

job A set of commands or processes executed by a batch system.

journaling In a file system that features a write transaction log, logging of write activities for replay across actual file-system structures and consistency protection.

journaling file system A file system that features a write transaction log where all write activities are logged for replay across actual file-system structures and consistency protection.

just-in-time (JIT) In Java virtual machine implementations, describes a compiler that converts bytecode to native CPU instructions the first time the bytecode is interpreted to speed later execution.

Kerberos A network authentication protocol invented at M.I.T. that forms the basis for the Microsoft network authentication protocol.

kernel The operating system component running on the computer at all times after system boot.

kernel abstractions Components provided with the Mach microkernel to add functionality beyond the microkernel, such as tasks, threads, memory objects, and ports.

kernel environment In the layered macOS and iOS operating system design, the Darwin layer that includes the Mach microkernel and the BSD UNIX kernel.

kernel extensions (kexts) Third-party components added to the kernel (usually to support third-party devices or services).

kernel mode A CPU mode in which all instructions are enabled. The kernel runs in this mode. See also user mode.

kernel threads Threads running in kernel mode.

kernel-mode driver framework (KMDF) A framework in Windows to facilitate the writing of kernel-mode device drivers.

kernel-mode thread (KT) In Windows, the name for the state of a thread when it is running in kernel mode.

Kernighan's Law "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."

keys In the context of protection, unique bit patterns held by domains corresponding with unique bit patterns (locks) held by objects. Generally, secrets used in cryptography.

keystream An infinite set of bits used to encrypt a plain-text stream through an XOR operation in a stream cipher.

keystroke logger A program that captures keystrokes entered by users.

kilobyte (KB) 1,024 bytes.

Kubernetes An orchestration tool for containers.

labels In mandatory access control, identifiers assigned to objects and/or subjects. The label is checked by the operating system when an operation is requested to determine if it is allowed.

layered approach A kernel architecture in which the operating system is separated into a number of layers (levels); typically, the bottom layer (layer 0) is the hardware, and the highest (layer N) is the user interface.

lazy swapper A swapping method in which only pages that are requested are brought from secondary storage into main memory.

least frequently used (LFU) In general, an algorithm that selects the item that has been used least frequently. In virtual memory, when access counts are available, selecting the page with the lowest count.

least privilege Design principle stating that every program and every privileged user of the system should operate using the least amount of privilege necessary to complete the job.

least recently used (LRU) In general, an algorithm that selects the item that has been used least recently. In memory management, selecting the page that has not been accessed in the longest time.

lggroups In Solaris, locality groups located in the kernel; each lggroup gathers together CPUs and memory, and each CPU in that group can access any memory in the group within a defined latency interval. A method for dealing with NUMA.

library operating systems The applications that run on unikernels, containing both the kernel and the application code.

lightweight directory-access protocol (LDAP) A secure distributed naming service used throughout the computer industry.

lightweight process (LWP) A virtual processor-like data structure allowing a user thread to map to a kernel thread.

limit register A CPU register that defines the size of the range. Together with the base register, it defines the logical address space.

line discipline In Linux, an interpreter for the information from a terminal device.

link In file naming, a file that has no contents but rather points to another file.

linked allocation A type of file-system block allocation in which each file is a linked list of allocated blocks containing the file's contents. Blocks may be scattered all over the storage device.

linked list A data structure in which items are linked to one another.

linker A system service that combines relocatable object files into a single binary executable file.

Linux distribution A Linux system plus administrative tools to simplify the installation, upgrading, and management of the system.

Linux instance A set of Pico processes running in Windows created by WSL containing an init process and a bash shell (allowing executing of Linux commands within Windows)

Linux kernel The operating-system kernel of a Linux system.

Linux system The kernel, programs, and files that comprise a complete, runnable Linux system.

list A data structure that presents a collection of data values as a sequence.

little-endian A system architecture that stores the least significant byte first in a sequence of bytes.

Little's formula A scheduling equation ($n = ? \times W$) that is particularly useful because it is valid for any scheduling algorithm and arrival distribution.

live migration In virtualization, the movement of a running guest between two separate physical hosts.

LiveCD An operating system that can be booted and run from a CD-ROM (or more generally from any media) without being installed on a system's boot disk(s).

LiveDVD An operating system that can be booted and run from a DVD (or more generally from any media) without being installed on a system's boot disk(s).

livelock A condition in which a thread continuously attempts an action that fails.

liveness A set of properties that a system must satisfy to ensure that processes make progress during their execution life cycle.

living document A document that is modified over time to keep it up to date.

load balancing The movement of jobs or network packets between various components (say, computers in a network) to distribute the load or route around failures. Load balancing attempts to keep the workload evenly distributed across all processors in an SMP system.

load sharing The ability of a system with multiple CPU cores to schedule threads on those cores.

loadable kernel module (LKM) A kernel structure in which the kernel has a set of core components and can link in additional services via modules, either at boot time or during run time.

loader A system service that loads a binary executable file into memory, where it is eligible to run on a CPU core.

local procedure call In Windows, a method used for communication between two processes on the same machine.

local replacement In virtual-memory frame allocation, allowing a process to select a replacement frame only from the set of all frames allocated to the process.

local replacement algorithm A virtual-memory page replacement algorithm that avoids thrashing by not allowing a process to steal frames from other processes.

local-area network (LAN) A network that connects computers within a room, a building, or a campus.

locality The tendency of processes to reference memory in patterns rather than randomly.

locality model A model for page replacement based on the working-set strategy.

locality of reference The tendency of processes to reference memory in patterns rather than randomly.

location independence In distributed computing, a feature in which the name of an object does not need to be changed when the object's physical location changes.

location transparency In distributed computing, a feature in which the name of an object does not reveal its physical location.

lock A mechanism that restricts access by processes or subroutines to ensure integrity of shared data.

locked In general, fixed in place. In memory management, pages can be locked into memory to prevent them from being paged out.

lock-free An algorithm that provides protection from race conditions without requiring the overhead of locking.

locking Protecting critical sections, data structures, or objects through the use of code that coordinates access.

lock-key scheme In protection, a compromise between access lists and capability lists in which each object has a unique bit pattern (a lock) and each domain has a unique bit pattern (a key).

log file A file containing error or “logging” information; used for debugging or understanding the state or activities of the system.

log-based transaction-oriented file system A file system that features a write transaction log where all write activities are logged for replay across actual file-system structures and consistency protection.

logic bomb A remote-access tool designed to operate only when a specific set of logical conditions is met.

logical address Address generated by the CPU; must be translated to a physical address before it is used.

logical address space The set of all logical addresses generated by a program.

logical blocks Logical addresses used to access blocks on storage devices.

logical cluster numbers In Windows, the name given to secondary storage physical addresses.

logical file system A logical layer of the operating system responsible for file and file-system metadata management; maintains the FCBs.

logical formatting The creation of a file system in a volume to ready it for use.

logical memory Memory as viewed by the user; usually a large uniform array, not matching physical memory in virtual memory systems.

logical records File contents logically designated as fixed-length structured data.

LOOK An HDD I/O scheduling algorithm modification of SCAN that stops the head after the last request is complete (rather than at the innermost or outermost cylinder).

loopback Communication in which a connection is established back to the sender.

loosely coupled Describes a kernel design in which the kernel is composed of components that have specific and limited functions.

lottery scheduling A scheduling algorithm in which “lottery tickets” are given to threads and a lottery number is chosen at random to determine the next thread to get CPU time.

low-fragmentation heap (LFH) An optimization of the Windows default heap designed to decrease fragmentation.

low-level formatting The initialization of a storage medium in preparation for its use as a computer storage device.

Lustre A common open-source clustered file system.

LXC A Linux container technology.

Mach An operating system with microkernel structure and threading; developed at Carnegie Mellon University.

Mach-O The macOS format of executable files.

magic cookie A crude method of storing a text string at the start of a text file to indicate the type of text in the file.

magic number A crude method of storing a number at the start of a file to indicate the type of the data in the file.

magnetic tape A magnetic media storage device consisting of magnetic tape spooled on reels and passing over a read-write head. Used mostly for backups.

main queue Apple OS per-process block queue.

main TLB ARM CPU outer-level TLB; checked after the micro TLB lookup and before a miss causes a page table walk.

mainframe The largest class of computers (along with supercomputers), hosting hundreds of users and many and/or large jobs.

major fault In virtual memory, a page fault that can be resolved without having to page in data from secondary storage.

malware Software designed to exploit, disable, or damage computer systems.

mandatory access control (MAC) Access control settings enforced in the form of system policy.

mandatory file-lock mechanism A file-locking system in which the operating system enforces locking and file access.

man-in-the-middle attack An attack in which the attacker sits in the middle of the data flow of a communication, masquerading as the sender to the receiver and vice versa.

MapReduce A Google-created big data programming model and implementation for parallel processing across nodes in a distributed cluster. A layer on top of the Google file system (GFS), it

allows developers to carry out large-scale parallel computations easily.

marshaling Packaging a communication into an expected format for transmittal and reception.

maskable Describes an interrupt that can be delayed or blocked (such as when the kernel is in a critical section).

masquerading A practice in which a participant in a communication pretends to be someone else (another host or another person).

master boot record (MBR) Windows boot code, stored in the first sector of a boot partition.

master file directory (MFD) In two-level directory implementation, the index pointing to each UFD.

master file table The NTFS volume control block.

master key In the lock-key protection scheme, a key that is associated with each object and can be defined or replaced with the set-key operation to revoke or change access rights.

matchmaker A function that matches a caller to a service being called (e.g., a remote procedure call attempting to find a server daemon).

mean time between failure (MTBF) The statistical mean time that a device is expected to work correctly before failing.

mean time to data loss The statistical mean of the time until data is lost.

mean time to repair The statistical mean of the time to repair a device (e.g., to get a replacement and install it).

mechanical storage device A storage device based on moving mechanical parts (such as HDDs, optical disks, and magnetic tape); one form of nonvolatile storage.

mechanism An operation that defines how something will be done.

medium access control (MAC) address A unique byte number assigned to every Ethernet device allowing it to be located by packets sent across a LAN.

megabyte (MB) $1,024^2$ bytes.

memory Volatile storage within a computer system.

memory barriers Computer instructions that force any changes in memory to be propagated to all other processors in the system.

memory compression In memory management, an alternative to paging involving compressing the contents of frames to decrease the memory used.

memory compression process In Windows 10, a process that maintains a working set of compressed standby pages.

memory fences Computer instructions that force any changes in memory to be propagated to all other processors in the system.

memory manager (MM) The Windows name for the component that manages memory.

memory mapping A file-access method in which a file is mapped into the process memory space so that standard memory access instructions read and write the contents of the file; an alternative to the use of read() and write() calls.

memory model Computer architecture memory guarantee, usually either strongly ordered or weakly ordered.

memory resident Objects, such as pages, that are in main memory and ready for threads to use or execute.

memory stall An event that occurs when a thread is on CPU and accesses memory content that is not in the CPU's cache. The thread's execution stalls while the memory content is fetched.

memory transaction A sequence of memory read-write operations that are atomic.

memory-management unit (MMU) The hardware component of a computer CPU or motherboard that allows it to access memory.

memory-mapped file A file that is loaded into physical memory via virtual memory methods, allowing access by reading and writing to the memory address occupied by the file.

memory-mapped I/O A device I/O method in which device-control registers are mapped into the address space of the processor.

message In networking, a communication, contained in one or more packets, that includes source and destination information to allow correct delivery. In message-passing communications, a packet of information with metadata about its sender and receiver.

message digest The calculation resulting from a hash function.

message passing In interprocess communication, a method of sharing data in which messages are sent and received by processes. Packets of information in predefined formats are moved between processes or between computers.

message-authentication code (MAC) A cryptographic checksum calculated in symmetric encryption; used to authenticate short values.

message-passing model A method of interprocess communication in which messages are exchanged.

metadata A set of attributes of an object. In file systems, e.g., all details of a file except the file's contents.

G-22 Glossary

metaslabs Chunks of blocks. In ZFS, a pool of storage is split into metaslabs for easier management.

methods In Java, functions that act on objects and data fields.

metropolitan-area network (MAN) A network linking buildings within a city.

micro TLB ARM CPU inner-level TLBs, one for instructions and one for data.

microkernel An operating-system structure that removes all nonessential components from the kernel and implements them as system and user-level programs.

Microsoft interface definition language The Microsoft text-based interface definition language; used, e.g., to write client stub code and descriptors for RPC.

middleware A set of software frameworks that provide additional services to application developers.

minicomputer A mid-sized computer, smaller than a mainframe but larger (in resources and users) than a workstation.

minidisks Virtual disks used in early IBM virtual systems.

minimum granularity In the Completely Fair Scheduler, a configurable variable representing the minimum length of time any process is allocated to the processor.

miniport driver In Windows I/O, the device-specific driver.

minor fault In virtual memory, a page fault resolved by executing an I/O to bring in the page from secondary storage.

mirrored volume A volume in which two devices are mirrored.

mirroring In storage, a type of RAID protection in which two physical devices contain the same content. If one device fails, the content can be read from the other.

mobile computing A mode of computing involving small portable devices like smartphones and tablet computers.

mode bit A CPU status bit used to indicate the current mode: kernel (0) or user (1).

modify bit An MMU bit used to indicate that a frame has been modified (and therefore must have its contents saved before page replacement).

module loader and unloader In Linux, user-mode utilities that work with the module-management system to load modules into the kernel.

module-management system In Linux, the facility that allows kernel modules to be loaded into

memory and to communicate with the rest of the kernel.

monitor A high-level language synchronization construct that protects variables from race conditions.

monitor call A software-triggered interrupt allowing a process to request a kernel service.

monoculture A community of computer systems that are very similar to one another. This similarity makes them easier to attack and thus represents a threat to security.

monolithic In kernel architecture, describes a kernel without structure (such as layers or modules).

Moore's Law A law predicting that the number of transistors on an integrated circuit would double every eighteen months.

most frequently used (MFU) In general, an algorithm that selects the item that has been used most frequently. In virtual memory, when access counts are available, selecting the page with the highest count.

mount point The location within the file structure where a file system is attached.

mount protocol The protocol for mounting a file system in a remote file system.

mount table An in-memory data structure containing information about each mounted volume. It tracks file systems and how they are accessed.

mounting Making a file system available for use by logically attaching it to the root file system.

multicore Multiple processing cores within the same CPU chip or within a single system.

multicore processor Multiple processing cores within the same CPU chip.

multicore systems Systems that have two or more hardware processors (CPU cores) in close communication, sharing the computer bus and sometimes the clock, memory, and peripheral devices.

multifactor authentication Authentication based on two or more sources of data, with more sources generally providing stronger authentication.

multilevel feedback queue A scheduling algorithm that allows a process to move between queues.

multilevel queue A scheduling algorithm that partitions the ready queue into several separate queues.

multiple universal-naming-convention provider (MUP) The component within Windows that executes remote file accesses.

multiple user interface (MUI) A Windows Vista feature that allows multiple user interfaces,

possibly configured for different locales, to be used concurrently.

multiprocessor Multiple processors within the same CPU chip or within a single system.

multiprocessor systems Systems that have two or more hardware processors (CPU cores) in close communication, sharing the computer bus and sometimes the clock, memory, and peripheral devices.

multiprogramming A technique that increases CPU utilization by organizing jobs (code and data) so that the CPU always has a job to execute.

multitasking The concurrent performance of multiple jobs. A CPU executes multiple jobs by switching among them, but the switches occur so frequently that users can interact with the processes.

multithreaded A term describing a process or program with multiple threads of control, allowing multiple simultaneous execution points.

mutex lock A mutual exclusion lock; the simplest software tool for assuring mutual exclusion.

mutual exclusion A property according to which only one thread or process can be executing code at once.

name server In the domain-name system, a host or software that provides resolving services.

named pipes A connection-oriented messaging mechanism—e.g., allowing processes to communicate within a single computer system.

named semaphore A POSIX scheduling construct that exists in the file system and can be shared by unrelated processes.

named shared-memory object In Windows API, a section of a memory-mapped file accessible by name from multiple processes.

namespace In Linux, a process's specific view of the file system hierarchy.

naming In distributed computing, the mapping between logical and physical objects.

national-language-support (NLS) A Windows API providing support for localization (including date, time, and money formats).

need-to-know principle The principle that only those resources currently needed should be available to use at a given time.

nested page tables (NPTs) In virtualization, a method used by the virtual machine manager to maintain page-table state both for guests and for the system.

network In the simplest terms, a communication path between two or more systems.

network address translation In networking, the mapping of one header address to another by modifying the network packets (e.g., allowing a host to provide IP addresses to multiple guests while presenting only one IP address to the connected network).

network computer A limited computer that understands only web-based computing.

network device interface specification (NDIS) An internal Windows networking interface separating network adapters from transport protocols.

network devices I/O devices that connect to a network.

network file system (NFS) A common network file system used by UNIX, Linux, and other operating systems to share files across a network.

network operating system A type of operating system that provides features such as file sharing across a network, along with a communication scheme that allows different processes on different computers to exchange messages. It provides an environment in which users can access remote resources by remote login or data transfer between remote and local systems.

network time protocol A network protocol for synchronizing system clocks.

network virtual memory A distributed computing feature similar to virtual memory but with the backing store on a remote system.

network-attached storage (NAS) Storage accessed from a computer over a network.

NFS protocol The protocol used for remote file access, remote file system mounting, etc., by the NFS file system.

nice value One of a range of values from -20 to +19, where a numerically lower value indicates a higher relative scheduling priority.

NIS A distributed naming service that provides username, password, hostname, and printer information to a set of computers.

nonblocking A type of I/O request that allows the initiating thread to continue while the I/O operation executes. In interprocess communication, a communication mode in which the sending process sends the message and resumes operation and the receiver process retrieves either a valid message or a null if no message is available. In I/O, a request that returns whatever data is currently available, even if it is less than requested.

noncontainer objects In Windows 10, a category of objects that cannot contain other objects.

nonmaskable interrupt An interrupt that cannot be delayed or blocked (such as an unrecoverable memory error)

nonpreemptive Scheduling in which, once a core has been allocated to a thread, the thread keeps the core until it releases the core either by terminating or by switching to the waiting state.

nonpreemptive kernels A type of kernel that does not allow a process running in kernel mode to be preempted; a kernel-mode process will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU.

nonrepudiation Proof that an entity performed an action (frequently performed by digital signatures).

non-uniform memory access (NUMA) An architecture aspect of many computer systems in which the time to access memory varies based on which core the thread is running on (e.g., a core interlink is slower than accessing DIMMs directly attached to core).

nonvolatile memory (NVM) Persistent storage based on circuits and electric charges.

nonvolatile storage (NVS) Storage in which data will not be lost in a power outage or similar event.

NOOP The Linux NVM scheduling algorithm, first come first served but with adjacent requests merged into fewer, larger I/O requests.

NUMA node One or more cores (e.g., cores that share a cache) that are grouped together as a scheduling entity for affinity or other uses.

NVM express (NVMe) A high-speed I/O bus for NVM storage.

NVRAM DRAM with battery or other backup power, rendering it nonvolatile.

object An instance of a class or an instance of a data structure. In Windows and generally, an instance of an object type.

object linking and embedding (OLE) A Microsoft technology allowing services to provide functions to components (e.g., for inserting spreadsheets into Word documents).

object manager In Windows, the kernel (executive) component that manipulates objects.

object type In Windows, a system-defined data type that has a set of attributes and methods that help define its behavior.

off-line Generally, a facility, system, or computing component that is unavailable. In file system implementation, operations executing while the file system is unavailable for use.

one-time password A password that is only valid once.

on-line Generally, a facility, system, or computing component that is available. In file system implementation, operations executing while the file system is available for use.

open count The number of processes having an open file.

open-file table An operating system data structure containing details of every file open within the system.

open-source operating system An operating system or other program available in source-code format rather than as compiled binary code.

operating system A program that manages a computer's hardware, provides a basis for application programs, and acts as an intermediary between the computer user and the computer hardware.

optimal page-replacement algorithm A theoretically optimal page replacement algorithm that has the lowest page-fault rate of all algorithms and never suffers from Belady's anomaly.

Orange Book U.S. Department of Defense Trusted Computer System Evaluation Criteria; a method of classifying the security of a system design.

orphan The child of a parent process that terminates in a system that does not require a terminating parent to cause its children to be terminated.

OS/2 A PC operating system from the mid 1980s co-developed by IBM and Microsoft to replace MS-DOS; generally considered to be a failure.

OSI protocol stack A set of cooperating network protocols that form a fully functional network. The OSI model formalized some of the earlier work done in network protocols but is currently not in widespread use.

out-of-band In networking, a term describing data delivered in a manner independent of the main data stream (e.g., delivery of a symmetric key in a paper document).

out-of-memory (OOM) killer In Linux, a routine that executes when free memory is very low, terminating processes to free memory.

over-allocating Generally, providing access to more resources than are physically available. In virtual memory, allocating more virtual memory than there is physical memory to back it.

overcommitment In virtualization, providing resources to guests that exceed available physical resources.

over-provisioning In non-volatile memory, space set aside for data writes that is not counted in the device free space.